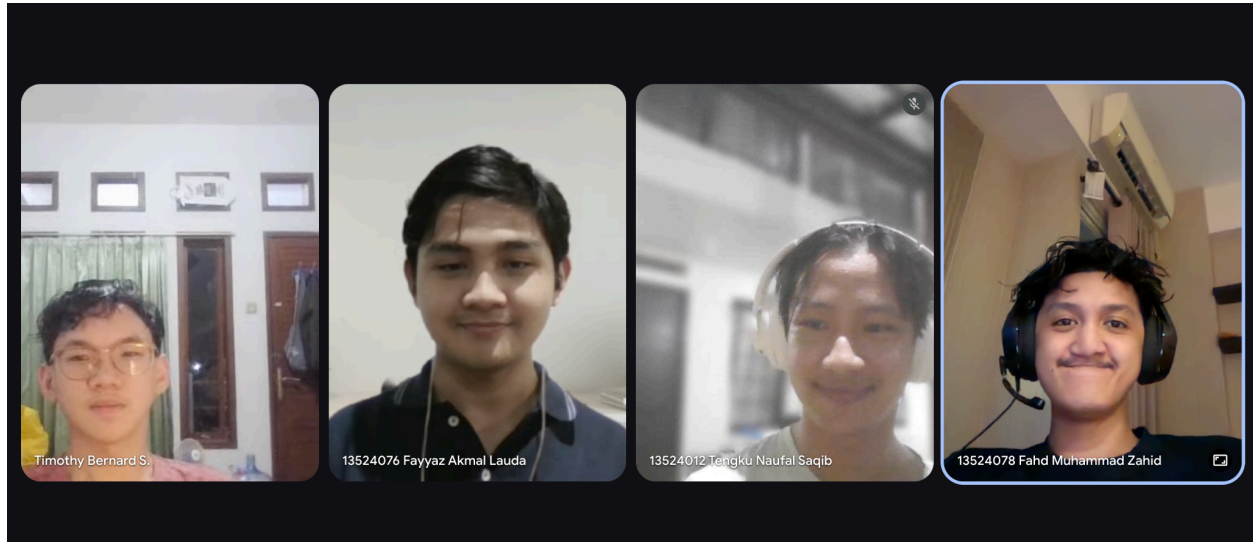


Laporan Tugas Besar
IF2224 Teori Bahasa Formal dan Otomata
Milestone 4 & 5 : Intermediate Code Generator and Interpreter



Disusun oleh Kelompok :

Fahd Muhammad Zahid	13524078
Timothy Bernard Soeharto	13524092
Fayyaz Akmal Lauda	13524076
Tengku Naufal Saqib	13524012

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG 2026

BAB I

Landasan Teori

1.1 Arsitektur Kompilator

Kompilator secara umum dibagi menjadi dua fase besar, yaitu *front-end* dan *back-end*. *Front-end* kompilator bertugas menganalisis kode sumber yang ditulis oleh programmer, meliputi analisis leksikal, sintaksis, dan semantik. Hasil dari fase ini adalah representasi internal program, seperti *Abstract Syntax Tree* (AST). Sementara itu, *back-end* kompilator bertugas mengubah representasi internal tersebut menjadi kode yang dapat dieksekusi, baik berupa *machine code*, *bytecode*, maupun *intermediate code*.

Pemisahan ini sangat menguntungkan karena bersifat modular. Jika suatu bahasa ingin diubah sintaksisnya, hanya *front-end* yang perlu dimodifikasi, sedangkan *back-end* dapat tetap digunakan selama format representasi internalnya tidak berubah. Sebaliknya, jika ingin menargetkan arsitektur mesin yang berbeda, hanya *back-end* yang perlu diganti.

1.2 Decorated AST

1.2.1 Pengertian Decorated AST

Abstract Syntax Tree (AST) adalah representasi hierarkis dari struktur program setelah melewati fase analisis sintaksis. AST berbeda dari *parse tree* biasa karena hanya menyimpan informasi yang relevan secara semantik, tanpa menyertakan detail sintaksis yang tidak penting seperti tanda kurung atau titik koma.

Setelah melewati fase analisis semantik (Milestone 3), AST tersebut "didekorasi" dengan informasi tambahan pada setiap node-nya. AST yang telah dianotasi ini disebut *Decorated AST* atau *Annotated AST*. Informasi tambahan yang umumnya melekat pada setiap node meliputi tipe data dari ekspresi pada node tersebut dan referensi ke entri tabel simbol yang sesuai.

1.2.2 Informasi yang Terkandung dalam Decorated AST

Terdapat dua jenis informasi utama yang melekat pada node *Decorated AST*:

- **Tipe Data**, setiap node yang merepresentasikan ekspresi memiliki anotasi tipe data, misalnya *integer*, *boolean*, atau *string*. Informasi ini dihasilkan dari proses *type checking* pada fase analisis semantik.
- **Referensi Tabel Simbol**, setiap node yang merepresentasikan sebuah variabel atau fungsi memiliki referensi ke entri yang sesuai di dalam tabel simbol. Referensi ini menyimpan informasi seperti ruang lingkup (*scope*) variabel dan alamat memorinya.

1.2.3 Peran Decorated AST

Decorated AST bersifat wajib sebagai masukan *Intermediate Code Generator* karena informasi anotasi yang terkandung di dalamnya digunakan untuk mengambil keputusan instruksi yang tepat. Sebagai contoh, ketika *generator* menemukan ekspresi $a + b$, informasi tipe data menentukan apakah operasi tersebut adalah penjumlahan *integer* atau penggabungan string (*string concatenation*). Tanpa anotasi ini, *generator* tidak dapat menentukan instruksi yang benar untuk dihasilkan.

1.3 Intermediate Code

1.3.1 Pengertian dan Fungsi Intermediate Code

Intermediate Code adalah representasi program yang berada di antara kode sumber dan kode mesin. *Intermediate code* bersifat independen terhadap mesin (*machine-independent*), sehingga memudahkan proses optimasi dan portabilitas kompilator ke berbagai platform. *Intermediate code* juga jauh lebih mudah dieksekusi dibandingkan AST karena berbentuk instruksi linier dan sekuensial, mirip dengan cara kerja CPU yang memproses instruksi satu per satu.

1.3.2 Three-Address Code (TAC)

Salah satu format *intermediate code* yang paling umum digunakan adalah *Three-Address Code (TAC)*. Dinamakan demikian karena setiap instruksi TAC hanya boleh melibatkan maksimal tiga alamat memori, umumnya terdiri dari dua operand sumber dan satu lokasi tujuan. Struktur dasar TAC adalah sebagai berikut:

$$hasil = operand_1 \operatorname{operator} operand_2$$

Karena setiap instruksi hanya mengerjakan satu operasi biner, ekspresi yang kompleks harus dipecah menjadi beberapa instruksi menggunakan variabel sementara (*temporary variables*), yang biasanya dinotasikan sebagai t_1 , t_2 , t_3 , dan seterusnya.

1.3.3 Translasi Ekspresi Aritmatika ke TAC

Proses translasi ekspresi aritmatika ke TAC dilakukan dengan cara menelusuri AST dari cabang terdalam (bottom-up) menuju akar. Setiap sub ekspresi dievaluasi terlebih dahulu dan hasilnya disimpan dalam variabel sementara sebelum digunakan oleh operasi di atasnya.

Sebagai contoh, ekspresi $x := (a + b) * (c - d)$ akan diratakan menjadi:

$$t_1 := a + b$$

$$t_2 := c - d$$

$$t_3 := t_1 * t_2$$

$$x := t_3$$

Dengan pendekatan ini, kompleksitas ekspresi bersarang berhasil diturunkan menjadi rangkaian instruksi primitif yang sederhana dan mudah dieksekusi.

1.3.4 Translasi Percabangan ke TAC

Struktur percabangan IF-ELSE pada kode sumber tidak dapat langsung direpresentasikan dalam bentuk linier. Solusinya adalah menggunakan kombinasi **Label** dan instruksi **Jump**. Terdapat dua jenis jump yang umum digunakan:

- **JPC (*Conditional Jump*)**, melompat ke suatu label hanya jika kondisi bernilai false.
- **JMP (*Unconditional Jump*)**, selalu melompat ke suatu label tanpa memedulikan kondisi apapun.

Contoh translasi IF-ELSE

```
Kode sumber: if x > 5 then y := 1 else y := 0

    LOD x
    LIT 5
    OPR GTR
    JPC L_ELSE
    LIT 1
    STO y
    JMP L_END
L_ELSE:
    LIT 0
    STO y
L_END:
```

Penggunaan JPC (*jump if false*) lebih umum dibandingkan *jump if true* karena memanfaatkan konsep *fall-through*, yaitu ketika kondisi bernilai benar, eksekusi langsung berlanjut ke instruksi berikutnya tanpa perlu lompatan tambahan.

1.3.5 Translasi Perulangan ke TAC

Perulangan WHILE memiliki kerumitan tambahan dibandingkan IF-ELSE karena setelah blok tubuh perulangan selesai dieksekusi, alur program harus kembali (jump back) ke bagian evaluasi kondisi.

Contoh translasi WHILE:

```
Kode sumber: while i < 5 do i := i + 1

L_START:
    LOD i
    LIT 5
    OPR LSS
    JPC L_END
    LOD i
    LIT 1
    OPR ADD
    STO i
    JMP L_START
L_END:
```

1.3.6 Daftar Instruksi yang Digunakan

Berikut adalah daftar instruksi TAC yang digunakan dalam bahasa Arion :

No	Instruksi	Nama	Keterangan
1	LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
2	LOD a	Load Value	Memuat nilai dari address a
3	STO a	Store Value	Menyimpan nilai ke address a
4	CAL l	Call	Memanggil fungsi di baris l
5	INT m	Initiate Memory	Membuat memori dengan ukuran m
6	JMP l	Unconditional Jump	Lompat ke baris l tanpa kondisi apapun
7	JPC l	Conditional Jump	Lompat ke baris l bila kondisi tidak terpenuhi
8	OPR o	Operation	Memanggil operasi o
9	RET	return	Keluar dari fungsi atau prosedur

1.3.7 Daftar Operasi OPR

Berikut adalah daftar operasi yang dapat dipanggil melalui instruksi OPR :

No	Nama	Fungsi
1	NEG	Negasi nilai teratas stack
2	ADD	Penjumlahan dua nilai teratas stack
3	SUB	Pengurangan dua nilai teratas stack
4	MUL	Perkalian dua nilai teratas stack
5	DIV	Pembagian dua nilai teratas stack
6	MOD	Modulus dua nilai teratas stack
7	EQL	Perbandingan sama dengan
8	NEQ	Perbandingan tidak sama dengan
9	LSS	Perbandingan kurang dari
10	GEQ	Perbandingan lebih dari atau sama dengan
11	GTR	Perbandingan lebih dari
12	LEQ	Perbandingan kurang dari atau sama dengan
13	WRT	Menulis output tanpa newline
14	WRTLN	Menulis output dengan newline

1.4 Interpreter dan Virtual Machine

1.4.1 Konsep Virtual Machine

Setelah *Intermediate Code* berhasil dibuat, diperlukan sebuah mesin untuk mengeksekusinya. Mesin ini disebut *Interpreter* atau *Virtual Machine* (VM). Sebuah interpreter membaca dan mengeksekusi instruksi satu per satu secara langsung, berbeda dengan kompilator yang mengubah seluruh program terlebih dahulu sebelum dieksekusi.

Dalam konteks ini, *Interpreter* bahasa Arion bertindak sebagai VM yang mensimulasikan perangkat keras komputer secara *software*, memiliki memori sendiri dan logika eksekusi instruksi.

1.4.2 Siklus Eksekusi: *Fetch-Decode-Execute*

Sama seperti CPU fisik, *Interpreter* beroperasi dalam siklus yang berulang yang terdiri dari tiga tahap:

- **Fetch**, Membaca instruksi TAC yang saat ini ditunjuk oleh Instruction Pointer (IP).
- **Decode**, Mengurai instruksi tersebut untuk mengetahui opcode dan operandnya.
- **Execute**, Melakukan aksi yang sesuai, misalnya operasi aritmatika, penyimpanan nilai, atau lompatan.

Setelah setiap tahap *Execute* selesai, IP bergerak maju ke instruksi berikutnya ($IP := IP + 1$), kecuali instruksi tersebut adalah instruksi jump yang akan mengubah nilai IP secara langsung. Siklus ini terus berulang hingga instruksi RET pada baris terakhir dieksekusi.

1.4.3 Stack Sebagai Memori Eksekusi

Struktur memori utama yang digunakan oleh *Interpreter* adalah *Stack*. *Stack* bekerja berdasarkan prinsip LIFO (*Last In, First Out*), artinya data yang terakhir dimasukkan adalah data yang pertama kali diambil. Dua operasi utama pada *Stack* adalah:

- **Push**, Menambahkan data ke puncak stack.
- **Pop**, Mengambil dan menghapus data dari puncak stack.

Stack digunakan karena sifatnya yang sangat sesuai dengan cara kerja pemanggilan fungsi yang bersarang (*nested function calls*), di mana setiap fungsi yang dipanggil membutuhkan ruang memori tersendiri yang bersifat sementara.

1.4.4 Stack Frame dan Scope

Setiap kali sebuah fungsi atau prosedur dipanggil, *Interpreter* membuat sebuah blok memori baru di atas stack yang disebut *Stack Frame*. *Stack Frame* ini menyimpan:

- **Static Link**, Referensi ke *frame* dari blok yang secara leksikal melingkupinya (*lexical parent*).
- **Dynamic Link**, Referensi ke *frame* pemanggil (*caller*).
- **Return Address**, Alamat instruksi yang harus dieksekusi setelah fungsi selesai.
- **Variabel Lokal**, Semua variabel yang dideklarasikan di dalam fungsi tersebut.

Ketika fungsi selesai dieksekusi, *frame*-nya di-*pop* dari stack dan semua variabel lokalnya otomatis hilang. Mekanisme inilah yang menjelaskan mengapa variabel lokal di satu fungsi tidak dapat diakses oleh fungsi lain.

Bab II

Perancangan dan Implementasi

2.1 Gambaran Umum Sistem

Pada Milestone 4, sistem dikembangkan sebagai tahap back-end dari compiler bahasa Arion. Tahap ini melanjutkan hasil Milestone 1 sampai Milestone 3. Milestone 1 menghasilkan token melalui lexer, Milestone 2 menyusun token menjadi parse tree melalui parser, dan Milestone 3 membangun AST serta melakukan analisis semantik untuk menghasilkan Decorated AST dan symbol table.

Masukan utama tahap ini adalah Decorated AST, yaitu AST yang setiap node-nya sudah memiliki informasi semantik seperti tipe data, lexical level, dan referensi ke symbol table. Informasi tersebut digunakan oleh Intermediate Code Generator untuk menghasilkan instruksi perantara yang bersifat linier. Instruksi perantara ini kemudian dijalankan oleh Interpreter atau Virtual Machine berbasis stack.

Secara umum, alur sistem adalah sebagai berikut.

1. Source Code Arion
2. Lexer
3. Parser
4. AST Builder
5. Semantic Analyzer
6. Decorated AST + Symbol Table
7. Intermediate Code Generator
8. Intermediate Code
9. Virtual Machine / Interpreter
10. Output Program

Dengan pendekatan ini, program Arion tidak langsung diterjemahkan menjadi executable native seperti program C/C++, melainkan diterjemahkan menjadi intermediate code yang dijalankan oleh virtual machine. Hal ini membuat eksekusi program lebih mudah dikontrol, terutama untuk pengelolaan memori, percabangan, perulangan, dan output.

2.2 Teknologi dan Bahasa Pemrograman

Implementasi Milestone 4 menggunakan bahasa C++, mengikuti bahasa yang digunakan pada milestone sebelumnya. C++ dipilih karena mendukung paradigma pemrograman prosedural dan berorientasi objek, sehingga cocok untuk membangun komponen compiler yang modular, seperti lexer, parser, semantic analyzer, code generator, dan virtual machine.

Proyek dikompilasi menggunakan g++ dengan standar C++11. Build program dikelola menggunakan Makefile agar setiap komponen dapat dikompilasi secara terpisah maupun bersama-sama. Struktur program tetap mempertahankan pemisahan modul dari milestone sebelumnya, kemudian menambahkan modul baru untuk code generation dan interpreter.

Beberapa fitur C++ yang digunakan dalam implementasi antara lain class dan struct untuk representasi node, instruksi, nilai runtime, dan memori VM; vector untuk menyimpan daftar instruksi, stack, dan slot memori; exception class untuk menangani runtime error; serta enum class untuk mendefinisikan opcode dan operasi OPR secara eksplisit.

2.3 Struktur Program

Struktur program dibagi berdasarkan tahapan compiler. Modul lexer berada pada folder src/lexer dan bertanggung jawab mengubah source code menjadi token. Modul parser berada pada folder src/parser dan bertugas membentuk parse tree. Modul semantic berada pada folder src/semantic dan bertugas membangun AST, melakukan pemeriksaan semantik, serta menghasilkan symbol table.

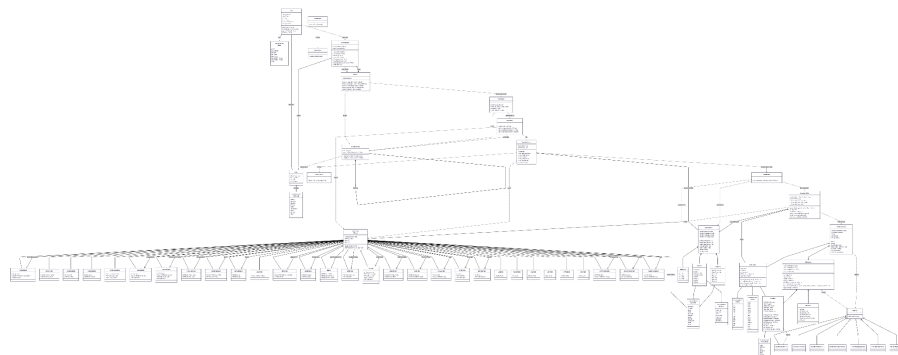
Pada Milestone 4, ditambahkan dua modul utama, yaitu src/codegen dan src/interpreter.

Folder src/codegen berisi komponen Intermediate Code Generator. File Instruction.hpp mendefinisikan format instruksi intermediate code, daftar opcode, dan daftar operasi OPR. File CodegenVisitor.hpp mendefinisikan interface visitor yang digunakan untuk menelusuri node-node AST dan menghasilkan daftar instruksi.

Folder src/interpreter berisi komponen Virtual Machine. File VM_Value.hpp mendefinisikan representasi nilai runtime seperti integer, real, boolean, char, dan string. File VM_Memory.hpp dan VM_Memory.cpp mengimplementasikan memori eksekusi berbasis stack dan slot memori. File VirtualMachine.hpp mendefinisikan interface mesin virtual yang menjalankan instruksi. File VM_Exceptions.hpp dan VM_Exceptions.cpp mendefinisikan exception runtime seperti stack underflow, invalid memory access, invalid operation, dan division by zero.

Dengan pembagian ini, setiap komponen memiliki tanggung jawab yang jelas. Code generator hanya bertugas menghasilkan instruksi, sedangkan interpreter hanya bertugas menjalankan instruksi tersebut.

2.3.1 Class Diagram



Untuk gambar lebih jelas dapat dilihat [disini](#)

Lexer & Token

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
-------	-------	---------------	--------------	-----------

Lexer	Class	source, pos, line, column, currentState, lexeme	tokenize(), getNextToken(), isAtEnd()	Kelas utama analisis leksikal yang memproses kode sumber karakter per karakter menggunakan mesin state untuk menghasilkan daftar token
Token	Class	type, value, line, column	—	Merepresentasikan satu unit leksikal hasil tokenisasi beserta informasi posisinya
TokenType	Enumeration	INTCON, REALCON, CHARCON, STRING, IDENT, PROGRAMSY, VARSY, BEGINSY, ENDSY	—	Mendefinisikan seluruh jenis token yang dikenali bahasa Arion
State	Enumeration	START, IN_INTEGER, IN_REAL, IN_IDENT, IN_STRING, IN_COMMENT_BRACE, IN_COMMENT_PAREN, DONE	—	Merepresentasikan kondisi mesin state di dalam Lexer untuk menentukan cara memproses karakter berikutnya

Parser & Parse Tree

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
TokenStream	Class	tokens, currentIndex	peek(), advance(), match(), expect(), isAtEnd()	Pembungkus daftar token yang menyediakan antarmuka navigasi sekuensial untuk Parser

Parser	Class	ts	parse_program(), parse_statement(), parse_expression(), parse_factor()	Melakukan analisis sintaksis dengan recursive descent parsing untuk membangun Parse Tree
ParseTreeNode	Class	name, children	addChild(), addToken()	Merepresentasikan satu simpul dalam Parse Tree yang membentuk struktur hierarkis program
TreePrinter	Class	—	printToConsole(), printToFile(), toString()	Kelas utilitas untuk menampilkan Parse Tree ke konsol atau file
SyntaxError	Class	—	SyntaxError(message)	Exception yang dilempar ketika Parser menemukan token yang tidak sesuai tata bahasa
ErrorHandler	Class	—	syntaxError(message)	Kelas pembantu yang memisahkan logika penanganan error dari logika parsing utama

Semantic AST Core

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
ASTNode	Abstract Class	type_code, tab_index, lev, line	node_type(), print()	Kelas abstrak basis seluruh simpul AST yang menyimpan informasi semantik hasil anotasi
TypeCode	Enumeration	UNKNOWN, INTEGER, REAL, CHAR, BOOLEAN, STRING, ARRAY, RECORD, SUBRANGE, VOID, ENUM	—	Mendefinisikan seluruh tipe data yang dikenali dalam bahasa Arion

ObjClass	Enumeration	CONSTANT, VARIABLE, TYPE, PROCEDURE , FUNCTION, RESERVED, PROGRAM	—	Mengklasifikasikan jenis entitas yang dapat terdaftar di tabel simbol
----------	-------------	---	---	---

Declaration Nodes

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
ProgramNode	Class	name, declarations, body	—	Merepresentasikan simpul akar program, menyimpan nama, seluruh deklarasi, dan blok utama
VarDeclNode	Class	names, type_node	—	Merepresentasikan deklarasi variabel beserta tipe datanya
ConstDeclNode	Class	name, value	—	Merepresentasikan deklarasi konstanta beserta nilai ekspresinya
TypeDeclNode	Class	name, type_def	—	Merepresentasikan deklarasi tipe buatan pengguna (user-defined type)
ParamGroupNode	Class	names, type_node, is_var_param	—	Merepresentasikan satu kelompok parameter fungsi/prosedur beserta penanda apakah dikirim per referensi atau nilai
ProcDeclNode	Class	name, params, body	—	Merepresentasikan deklarasi prosedur beserta parameter dan tubuhnya
FuncDeclNode	Class	name, params, return_type, body	—	Merepresentasikan deklarasi fungsi, identik dengan prosedur namun memiliki tipe kembalian

ArrayTypeNode	Class	index_range, elem_type	—	Merepresentasikan definisi tipe array dengan rentang indeks dan tipe elemennya
RecordTypeNode	Class	fields, is_anonymous	—	Merepresentasikan definisi tipe record beserta daftar field-nya
SubrangeNode	Class	low, high	—	Merepresentasikan tipe subrange sebagai rentang nilai antara batas bawah dan atas
EnumNode	Class	values	—	Merepresentasikan tipe enumerasi dengan daftar nama nilai yang didefinisikan pengguna

Statement Nodes

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
BlockNode	Class	statements, btabs_index	—	Merepresentasikan blok begin...end yang menyimpan daftar pernyataan sekuensial
AssignNode	Class	target, value	—	Merepresentasikan pernyataan assignment (:=) dengan simpul target dan simpul nilai
IfNode	Class	condition, then_branch, else_branch	—	Merepresentasikan percabangan IF-THEN-ELSE, cabang else bersifat opsional
WhileNode	Class	condition, body	—	Merepresentasikan perulangan WHILE-DO dengan kondisi dievaluasi di awal setiap iterasi

ForNode	Class	var_name, from_expr, to_expr, is_downto, body	—	Merepresentasikan perulangan FOR dengan variabel kontrol, rentang nilai, dan arah iterasi
RepeatNode	Class	body, condition	—	Merepresentasikan perulangan REPEAT-UNTIL dengan kondisi dievaluasi di akhir sehingga tubuh selalu dieksekusi minimal sekali
CaseNode	Class	selector, cases	—	Merepresentasikan percabangan CASE-OF dengan simpul selektor dan daftar pasangan nilai-cabang
ProcCallNode	Class	name, args	—	Merepresentasikan pemanggilan prosedur beserta daftar argumennya

Expression Nodes

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
BinOpNode	Class	op, left, right	—	Merepresentasikan operasi biner seperti +, -, *, /, dan operator perbandingan
UnaryOpNode	Class	op, operand	—	Merepresentasikan operasi unary seperti negasi dan not
VarNode	Class	name	—	Merepresentasikan penggunaan variabel dalam ekspresi
NumberNode	Class	value	—	Merepresentasikan literal bilangan bulat
RealNode	Class	value	—	Merepresentasikan literal bilangan real

CharNode	Class	value	—	Merepresentasikan literal karakter tunggal
StringNode	Class	value	—	Merepresentasikan literal string
BoolNode	Class	value	—	Merepresentasikan literal boolean (true atau false)
ArrayAccessNode	Class	array, indices	—	Merepresentasikan pengaksesan elemen array, mendukung array multidimensi
RecordAccessNode	Class	record, field_name	—	Merepresentasikan pengaksesan field record menggunakan operator titik (.)
FuncCallExprNode	Class	name, args	—	Merepresentasikan pemanggilan fungsi dalam ekspresi yang menghasilkan nilai kembalian

Semantic Analysis

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
ASTBuilder	Class	—	build(), build_program(), build_statement(), build_expression()	Mengonsumsi Parse Tree dan mengubahnya menjadi AST dengan method pembangun per jenis simpul
SemanticVisitor	Class	sym, inside_loop	visit(), visit_program(), visit_assign(), visit_if(), visit_while(), visit_binop(), visit_var()	Menelusuri AST menggunakan pola visitor untuk melakukan pengecekan tipe, resolusi nama, dan menghasilkan Decorated AST

SymbolTable	Class	tab, btab, atab, scope_stack	enter_tab(), lookup(), open_scope(), close_scope(), print_all()	Struktur data sentral yang menyimpan seluruh entitas yang dideklarasikan, diorganisasi dalam tiga tabel terpisah
TabEntry	Class	id, link, obj, type, ref, nrm, lev, adr	—	Satu entri tabel simbol utama yang menyimpan nama, kelas objek, tipe data, level scope, dan alamat memori suatu entitas
BtabEntry	Class	last, lpar, psze, vsze	—	Satu entri tabel blok yang menyimpan informasi scope termasuk ukuran area parameter dan variabel lokal
AtabEntry	Class	xtyp, etyp, eref, low, high, elsz, size	—	Satu entri tabel array yang menyimpan tipe indeks, tipe elemen, batas indeks, dan total ukuran array
SemanticError	Class	—	SemanticError(message, line)	Exception yang dilempar saat ditemukan pelanggaran semantik seperti variabel tidak dideklarasikan atau ketidakcocokan tipe

Code Generation

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
DASTReader	Class	—	load_decorated_ast(text, sym)	Membaca file teks Decorated AST dan merekonstruksi kembali ASTNode beserta SymbolTable

				ke dalam memori
CodegenVisitor	Class	instructions_, sym_, current_func_tab_index_, current_func_result_addr_, current_func_level_	generate(), emit(), patch_operand(), emit_lvalue_address(), ast_op_to_opr()	Kelas utama Intermediate Code Generator yang menelusuri Decorated AST dan menghasilkan daftar instruksi TAC, termasuk mekanisme back-patching untuk instruksi jump
Instruction	Class	op, level, operand, literal_value, has_literal	make(), literal(), operation(), to_string()	Merepresentasikan satu instruksi TAC dengan opcode, level scope, dan operandnya
OpCode	Enumeration	LIT, LOD, LDA, LDI, STI, STO, CAL, INT, JMP, JPC, OPR, RET	—	Mendefinisikan seluruh opcode instruksi TAC yang digunakan dalam bahasa Arion

OprCode	Enumeration	NEG, ADD, SUB, MUL, DIV, MOD, IDIV, EQL, NEQ, LSS, GTR, AND, OR, WRT, WRTLN	—	Mendefinisikan seluruh kode operasi untuk instruksi OPR, mencakup aritmatika, logika, perbandingan, dan output
---------	-------------	---	---	--

Virtual Machine / Interpreter

Kelas	Jenis	Atribut Utama	Method Utama	Deskripsi
VirtualMachine	Class	code_, memory_, ip_, halted_, output_	run(), step(), is_halted(), set_code(), execute()	Kelas utama Interpreter yang mengelola siklus Fetch-Decode-Execute dan mendelegasikan manajemen memori ke VMMemory
VMMemory	Class	slots_, stack_, frames_, current_base_	push(), pop(), peek(), load(), store(), push_frame(), pop_frame()	Mengelola seluruh memori eksekusi program termasuk Stack, Stack Frame, dan pengalamatan variabel berdasarkan level scope

VMFrame	Class	base, static_link, dynamic_link, return_address, previous_base, previous_memory_size, level	—	Merepresentasikan satu Stack Frame yang menyimpan konteks eksekusi fungsi/prosedur yang sedang berjalan
VMValue	Class	type_, int_value_, real_value_, bool_value_, char_value_, string_value_	integer(), real(), boolean(), character(), as_int(), as_real(), as_bool(), to_output_string()	Merepresentasikan satu nilai dalam Stack yang dapat berupa berbagai tipe data menggunakan pola tagged union
VMValueType	Enumeration	NONE, INTEGER, REAL, BOOLEAN, CHAR, STRING	—	Mendefinisikan seluruh tipe nilai yang dapat disimpan dalam VMValue
VMError	Class	—	VMError(message)	Kelas basis seluruh exception yang dapat terjadi selama eksekusi Virtual Machine
StackOverflowError	Class	—	—	Dilempar ketika Stack melebihi batas

				maksimum, umumnya akibat rekursi tak terbatas
StackUnderflowError	Class	—	—	Dilempar ketika operasi pop dilakukan pada Stack yang sudah kosong
InvalidMemoryAccessError	Class	—	—	Dilempar ketika terjadi akses memori di luar rentang valid, misalnya indeks array di luar batas
InvalidJumpTargetError	Class	—	—	Dilempar ketika instruksi jump menuju baris instruksi yang tidak ada
InvalidOperationError	Class	—	—	Dilempar ketika instruksi OPR menerima kode operasi yang tidak dikenali
DivisionByZeroError	Class	—	—	Dilempar ketika terjadi operasi pembagian dengan nilai nol

TypeMismatchError	Class	—	—	Dilempar ketika operasi dilakukan pada nilai dengan tipe data yang tidak kompatibel
-------------------	-------	---	---	--

2.4 Intermediate Code Generator

Intermediate Code Generator adalah komponen yang mengubah Decorated AST menjadi daftar instruksi intermediate code. Instruksi yang digunakan mengikuti model stack machine, yaitu operasi dilakukan dengan cara memuat operand ke stack, menjalankan operasi terhadap nilai teratas stack, lalu menyimpan hasilnya kembali ke memori jika diperlukan.

Instruksi utama yang digunakan adalah LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, dan RET. Instruksi LIT digunakan untuk memasukkan literal ke stack. LOD digunakan untuk memuat nilai dari alamat memori. STO digunakan untuk menyimpan nilai ke alamat memori. INT digunakan untuk menginisialisasi ruang memori. JMP dan JPC digunakan untuk mengatur alur kontrol. CAL dan RET digunakan untuk pemanggilan dan pengembalian fungsi/prosedur. OPR digunakan untuk operasi aritmatika, perbandingan, dan output.

2.4.1 Pendekatan Traversal

Code generator menggunakan pendekatan traversal terhadap AST. Karena AST berbentuk pohon, proses translasi dilakukan dengan mengunjungi node dari akar ke anak-anaknya sesuai struktur program. Untuk statement, traversal dilakukan secara top-down mengikuti urutan eksekusi program. Untuk expression, traversal dilakukan secara post-order, yaitu operand kiri dan kanan diproses terlebih dahulu sebelum operatornya.

Sebagai contoh, pada ekspresi $a + b$, generator terlebih dahulu menghasilkan instruksi untuk memuat nilai a , lalu memuat nilai b , kemudian menghasilkan instruksi OPR ADD. Dengan demikian, ketika instruksi ADD dijalankan oleh VM, dua nilai yang dibutuhkan sudah berada di stack.

Pola traversal ini cocok dengan struktur AST karena setiap node dapat diterjemahkan berdasarkan jenisnya. AssignNode diterjemahkan menjadi instruksi evaluasi ekspresi diikuti STO. IfNode diterjemahkan menjadi evaluasi kondisi diikuti JPC dan JMP. WhileNode diterjemahkan menjadi label awal, evaluasi kondisi, JPC ke akhir loop, body loop, dan JMP kembali ke awal.

2.4.2 Pembacaan dan Parsing Decorated AST

Decorated AST yang digunakan oleh code generator berasal dari tahap semantic analysis. Pada implementasi pipeline, AST tidak perlu diparsing kembali sebagai teks, melainkan

diteruskan sebagai struktur data hasil ASTBuilder dan SemanticVisitor. SemanticVisitor menambahkan informasi penting pada node, seperti type_code, tab_index, dan lev.

Informasi tab_index digunakan untuk mencari entri identifier pada symbol table. Dari symbol table, code generator dapat mengetahui alamat variabel, tipe data, dan level scope. Informasi tipe digunakan untuk menentukan operasi yang sesuai, sedangkan informasi alamat digunakan untuk menghasilkan instruksi LOD dan STO.

Dengan demikian, code generator tidak hanya membaca bentuk sintaks program, tetapi juga memanfaatkan hasil analisis semantik. Hal ini penting karena dua ekspresi dengan bentuk sintaks yang sama dapat memerlukan perlakuan berbeda jika tipe datanya berbeda.

2.4.3 Inisiasi Memori

Sebelum statement utama dieksekusi, code generator menghasilkan instruksi INT untuk mengalokasikan ruang memori awal. Besar memori ditentukan berdasarkan jumlah variabel dan kebutuhan stack frame. Pada model yang digunakan, tiga slot pertama pada frame disediakan untuk informasi frame, yaitu static link, dynamic link, dan return address.

Konvensi alamat yang digunakan adalah sebagai berikut.

Alamat 0: static link

Alamat 1: dynamic link

Alamat 2: return address

Alamat 3 dan seterusnya: variabel

Jika symbol table menyimpan variabel x dengan adr = 0, maka alamat runtime variabel tersebut menjadi 3. Jika variabel y memiliki adr = 1, maka alamat runtime-nya menjadi 4. Dengan konvensi ini, instruksi yang dihasilkan tetap konsisten dengan struktur stack frame.

2.4.4 Translasi Assignment dan Ekspresi

Assignment diterjemahkan dengan cara mengevaluasi ekspresi di sisi kanan terlebih dahulu, lalu menyimpan hasilnya ke alamat variabel tujuan. Misalnya statement $x := 10$ diterjemahkan menjadi:

```
LIT 0 10
STO 0 3
```

Instruksi LIT memasukkan nilai 10 ke stack. Instruksi STO mengambil nilai teratas stack dan menyimpannya ke alamat variabel x.

Untuk ekspresi biner seperti $x + 5$, generator menghasilkan instruksi untuk memuat operand kiri, memuat operand kanan, lalu menjalankan operasi. Contohnya:

```
LOD 0 3
```

```
LIT 0 5
OPR 0 2
```

Pada contoh tersebut, OPR 0 2 merepresentasikan operasi ADD. Hasil operasi akan diletakkan kembali di stack sehingga dapat digunakan oleh instruksi berikutnya, misalnya STO untuk assignment atau OPR WRTLN untuk output.

2.4.5 Pemetaan Operator dan Ekspresi Unari

Pada evaluasi ekspresi, code generator menelusuri node secara post-order sehingga operand selalu siap di stack sebelum operator dieksekusi. Pemetaan operator dari teks AST menjadi kode operasi OPR ditangani secara langsung. Operator aritmetika standar dipetakan secara ekuivalen (contoh: + menjadi ADD, * menjadi MUL). Terdapat perbedaan pada operasi pembagian: pembagian bilangan real (/) dipetakan ke DIV, sedangkan pembagian bilangan bulat (div) dipetakan secara terpisah menjadi IDIV.

Untuk ekspresi unari, terdapat penanganan khusus, terutama untuk operasi negasi dan logika. Unari minus (-x) diterjemahkan dengan memuat nilai operand ke stack, kemudian memanggil operasi NEG yang secara otomatis menegasikan nilai teratas stack.

Contoh translasi -x:

```
LOD 0 3
OPR 0 1
```

Sedangkan untuk unari logika (not flag), karena virtual machine tidak memiliki opcode khusus untuk operator not, operasi ini direkayasa menggunakan komparasi kesamaan dengan nilai false.

Contoh translasi not flag:

```
LOD 0 4
LIT 0 false
OPR 0 7
```

Pada contoh tersebut, nilai flag dimuat ke stack, diikuti oleh literal false, dan dikomparasi menggunakan instruksi EQL (OPR 0 7) yang ekuivalen dengan menghasilkan nilai kebalikan dari boolean aslinya.

2.4.6 Translasi Akses Array dan Pengalamatan Tak Langsung

Berbeda dengan variabel skalar yang alamatnya konstan, pengaksesan elemen array memerlukan kalkulasi alamat secara dinamis saat runtime (indirect addressing). Proses ini memanfaatkan instruksi LDA (Load Address) untuk memuat alamat basis ke stack, dan instruksi LDI (Load Indirect) atau STI (Store Indirect) untuk membaca atau menulis nilai pada alamat yang telah dikalkulasi.

Kalkulasi alamat elemen array melibatkan pengurangan indeks dengan batas bawah array (lower bound) dan perkalian dengan ukuran elemen.

Sebagai contoh, akses nilai `arr[i]` diterjemahkan menjadi:

```

LDA 0 5
LOD 0 3
LIT 0 1
OPR 0 3
OPR 0 2
LDI 0 0

```

Instruksi LDA 0 5 memuat alamat basis array ke stack. Indeks i kemudian dimuat (LOD 0 3) dan dikurangi dengan batas bawah array menggunakan LIT 0 1 dan OPR 0 3 (SUB). Offset yang dihasilkan dijumlahkan dengan alamat basis menggunakan instruksi ADD (OPR 0 2). Setelah alamat absolut elemen berada di puncak stack, instruksi LDI 0 0 mengeksekusi pembacaan nilai dari alamat tersebut.

2.4.7 Translasi Akses Record

Struktur pengaksesan field pada record menggunakan paradigma pengalamatan tak langsung yang serupa dengan array. Namun, offset untuk setiap field pada record bersifat statis dan telah dievaluasi sejak fase semantic analysis, sehingga tidak memerlukan kalkulasi indeks dinamis.

Sebagai contoh, ekspresi p.y (dengan asumsi variabel p memiliki alamat basis 3 dan field y memiliki offset statis 1) diterjemahkan menjadi:

```

LDA 0 3
LIT 0 1
OPR 0 2
LDI 0 0

```

Code generator memuat alamat basis record menggunakan LDA 0 3, kemudian langsung menambahkan offset statis field tersebut dengan memuat LIT 0 1 lalu menjalankan ADD (OPR 0 2). Nilai field kemudian diambil dari memori menggunakan instruksi LDI 0 0.

2.4.8 Pemanggilan Fungsi sebagai Ekspresi

Fungsi yang mengembalikan nilai (function) diterjemahkan serupa dengan prosedur (procedure) melalui instruksi CAL dan RET. Namun, dalam konteks evaluasi ekspresi, pemanggilan fungsi dipastikan meninggalkan hasil komputasinya (return value) di puncak stack evaluasi.

Seluruh argumen dievaluasi dari kiri ke kanan sehingga tersusun berurutan di atas stack. Setelah eksekusi CAL melompat ke blok fungsi dan kembali melalui RET, virtual machine memastikan bahwa frame lokal fungsi dihapus dan nilai kembalian diletakkan sebagai elemen teratas stack. Nilai kembalian ini kemudian langsung digunakan oleh operasi hierarki di atasnya tanpa instruksi perantara tambahan.

2.4.9 Translasi IF-ELSE

Statement IF-ELSE diterjemahkan menggunakan instruksi JPC dan JMP. Pertama, kondisi dievaluasi hingga menghasilkan nilai boolean di stack. Setelah itu, instruksi JPC digunakan untuk melompat ke bagian else jika kondisi bernilai false.

Pola umum translasi IF-ELSE adalah:

```
kode kondisi
JPC 0 alamat_else
kode then
JMP 0 alamat_akhir
kode else
alamat_akhir
```

Target JPC dan JMP biasanya belum diketahui saat instruksi pertama kali dibuat. Oleh karena itu, code generator menggunakan mekanisme backpatching, yaitu membuat instruksi dengan operand sementara, lalu memperbarui operand tersebut setelah alamat tujuan diketahui.

Dengan cara ini, struktur IF-ELSE yang semula bercabang pada AST dapat diratakan menjadi daftar instruksi linier.

2.4.10 Translasi WHILE

Statement WHILE diterjemahkan dengan menyimpan alamat awal loop, mengevaluasi kondisi, lalu menghasilkan JPC ke alamat akhir loop jika kondisi false. Jika kondisi true, body loop dieksekusi, kemudian instruksi JMP digunakan untuk kembali ke alamat awal loop.

Pola umum translasi WHILE adalah:

```
alamat_awal:
kode kondisi
JPC 0 alamat_akhir
kode body
JMP 0 alamat_awal
alamat_akhir:
```

Sebagai contoh, `while y > 0 do y := y - 1` diterjemahkan dengan memuat nilai y, memuat literal 0, menjalankan operasi GTR, lalu menggunakan JPC untuk keluar dari loop jika hasilnya false. Setelah body selesai, JMP mengembalikan instruction pointer ke awal pengecekan kondisi.

2.4.11 Translasi Fungsi/Prosedur

Fungsi dan prosedur diterjemahkan menggunakan instruksi CAL dan RET. Instruksi CAL digunakan untuk berpindah ke alamat awal fungsi atau prosedur, sedangkan RET digunakan untuk mengembalikan eksekusi ke alamat pemanggil.

Sebelum pemanggilan dilakukan, argumen fungsi atau prosedur dievaluasi dan disiapkan pada stack atau frame sesuai konvensi pemanggilan. Ketika CAL dieksekusi, VM membuat stack frame baru yang berisi static link, dynamic link, return address, parameter, dan variabel lokal. Setelah prosedur atau fungsi selesai, RET akan menghapus frame tersebut dan mengembalikan eksekusi ke return address.

Untuk fungsi yang menghasilkan nilai, hasil fungsi diletakkan kembali pada stack agar dapat digunakan sebagai bagian dari ekspresi lain. Untuk prosedur, tidak ada nilai yang dikembalikan, sehingga pemanggilan hanya menghasilkan efek samping seperti perubahan variabel atau output.

2.4.12 Translasi Write/Writeln

Write dan writeln ditangani sebagai pemanggilan prosedur bawaan. Setiap argumen dievaluasi terlebih dahulu sehingga nilainya berada di stack. Setelah itu, generator menghasilkan instruksi OPR WRT atau OPR WRTLN.

Instruksi OPR 0 13 digunakan untuk write, yaitu mencetak nilai tanpa newline. Instruksi OPR 0 14 digunakan untuk writeln, yaitu mencetak nilai disertai newline. Jika terdapat beberapa argumen, generator memproses argumen satu per satu sesuai urutan kemunculannya.

Contoh:

writeln(y)

diterjemahkan menjadi:

```
LOD 0 4
OPR 0 14
```

Instruksi LOD memuat nilai y ke stack, lalu OPR WRTLN mengambil nilai tersebut dan mencetaknya ke output.

2.4.13 Format Output Intermediate Code

Intermediate code dicetak dalam format bernomor agar mudah dibaca dan dapat digunakan oleh interpreter. Setiap baris instruksi memiliki bentuk umum:

nomor_instruksi OPCODE level operand

Contoh:

```
0 INT 0 5
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 LIT 0 5
```

```
5 OPR 0 11
6 JPC 0 12
```

Kolom pertama menunjukkan indeks instruksi. Opcode menunjukkan jenis operasi. Level digunakan untuk lexical level atau frame difference. Operand digunakan sesuai opcode, misalnya sebagai literal, alamat variabel, target jump, atau kode operasi OPR.

2.5 Stack Penyimpanan

Stack penyimpanan adalah komponen memori runtime yang digunakan oleh interpreter. Stack digunakan untuk menyimpan nilai sementara saat ekspresi dievaluasi, menyimpan nilai variabel, dan mengelola stack frame untuk fungsi atau prosedur.

Pada implementasi VM, memori dibagi menjadi slot memori dan stack evaluasi. Slot memori digunakan untuk menyimpan variabel dan informasi frame. Stack evaluasi digunakan untuk operasi sementara, misalnya ketika menjalankan LIT, LOD, OPR, dan STO.

2.5.1 Struktur Data Stack

Struktur data stack menggunakan prinsip LIFO, yaitu nilai yang terakhir dimasukkan akan menjadi nilai pertama yang diambil. Operasi push digunakan untuk memasukkan nilai ke puncak stack, sedangkan pop digunakan untuk mengambil nilai dari puncak stack.

Instruksi LIT dan LOD akan melakukan push ke stack. Instruksi OPR akan mengambil operand dari stack menggunakan pop, menjalankan operasi, lalu melakukan push terhadap hasilnya. Instruksi STO akan mengambil nilai dari stack menggunakan pop dan menyimpannya ke alamat memori tertentu.

Nilai runtime direpresentasikan menggunakan VMValue. VMValue dapat menyimpan beberapa tipe data dasar Arion, yaitu integer, real, boolean, char, dan string. Dengan representasi ini, stack dapat menyimpan berbagai jenis nilai tanpa kehilangan informasi tipe.

2.5.2 Manajemen Stack Frame

Stack frame digunakan untuk mengelola pemanggilan fungsi dan prosedur. Setiap frame memiliki tiga bagian utama pada awal frame, yaitu static link, dynamic link, dan return address. Static link digunakan untuk mengakses scope leksikal yang melingkupi fungsi. Dynamic link digunakan untuk kembali ke frame pemanggil. Return address menyimpan alamat instruksi yang harus dijalankan setelah fungsi atau prosedur selesai.

Ketika fungsi atau prosedur dipanggil, VM membuat frame baru dengan push_frame. Frame baru ini menjadi current frame sehingga akses variabel lokal dilakukan relatif terhadap base frame tersebut. Ketika fungsi selesai, pop_frame akan menghapus frame aktif dan mengembalikan base frame sebelumnya.

Mekanisme ini memungkinkan program memiliki fungsi atau prosedur yang dipanggil secara bersarang tanpa mencampur variabel lokal antar pemanggilan.

2.5.3 Pengalamatan Variabel

Pengalamatan variabel dilakukan berdasarkan informasi dari symbol table. Pada tahap semantic analysis, setiap variabel memiliki atribut adr yang menunjukkan offset variabel di dalam scope-nya. Code generator mengubah offset tersebut menjadi alamat runtime dengan menambahkan ukuran header stack frame.

Konvensi yang digunakan adalah:

$\text{alamat_runtime} = \text{adr} + 3$

Penambahan 3 dilakukan karena tiga slot pertama digunakan untuk static link, dynamic link, dan return address. Sebagai contoh, jika variabel x memiliki $\text{adr} = 0$, maka x disimpan pada alamat 3. Jika y memiliki $\text{adr} = 1$, maka y disimpan pada alamat 4.

Instruksi LOD menggunakan alamat ini untuk memuat nilai variabel ke stack, sedangkan STO menggunakan alamat ini untuk menyimpan nilai dari stack ke variabel.

2.6 Interpreter

Interpreter adalah komponen yang menjalankan intermediate code. Interpreter bekerja sebagai virtual machine yang memiliki instruction pointer, memori, stack evaluasi, dan daftar instruksi. Eksekusi dilakukan satu instruksi demi satu instruksi sampai program selesai.

Interpreter tidak membaca source code Arion secara langsung. Source code sudah diterjemahkan terlebih dahulu menjadi intermediate code oleh code generator. Dengan demikian, interpreter hanya perlu memahami instruksi VM seperti LIT, LOD, STO, JMP, JPC, OPR, dan RET.

2.6.1 Mekanisme Eksekusi Instruksi

Interpreter menggunakan siklus fetch-decode-execute. Pada tahap fetch, interpreter membaca instruksi pada posisi instruction pointer. Pada tahap decode, interpreter memeriksa opcode instruksi tersebut. Pada tahap execute, interpreter menjalankan aksi sesuai opcode.

Contohnya, jika opcode adalah LIT, interpreter akan memasukkan literal ke stack. Jika opcode adalah LOD, interpreter akan mengambil nilai dari memori dan memasukkannya ke stack. Jika opcode adalah STO, interpreter akan mengambil nilai dari stack dan menyimpannya ke memori. Jika opcode adalah JMP atau JPC, interpreter akan mengubah instruction pointer.

Secara normal, instruction pointer bertambah satu setelah sebuah instruksi selesai dieksekusi. Namun, untuk instruksi jump, nilai instruction pointer dapat berubah langsung ke target tertentu. Eksekusi berhenti ketika instruksi RET dijalankan atau ketika interpreter mencapai kondisi halt.

2.6.2 Penanganan OPR

Instruksi OPR digunakan untuk menjalankan operasi aritmatika, perbandingan, dan output. Operand pada instruksi OPR menunjukkan kode operasi yang harus dijalankan.

Daftar operasi yang digunakan adalah:

- 1 NEG
- 2 ADD
- 3 SUB
- 4 MUL
- 5 DIV
- 6 MOD
- 7 EQL
- 8 NEQ
- 9 LSS
- 10 GEQ
- 11 GTR
- 12 LEQ
- 13 WRT
- 14 WRTLN

Untuk operasi aritmatika seperti ADD, SUB, MUL, DIV, dan MOD, interpreter mengambil dua nilai teratas dari stack, menghitung hasilnya, lalu memasukkan hasil kembali ke stack. Untuk NEG, interpreter hanya mengambil satu nilai. Untuk operasi perbandingan seperti EQL, NEQ, LSS, GEQ, GTR, dan LEQ, interpreter menghasilkan nilai boolean. Untuk WRT dan WRTLN, interpreter mengambil nilai dari stack dan menambahkannya ke output.

2.6.3 Penanganan Jump dan Label

Jump digunakan untuk mengimplementasikan control flow. Instruksi JMP melakukan lompatan tanpa syarat ke indeks instruksi tertentu. Instruksi JPC melakukan lompatan bersyarat, yaitu hanya melompat jika nilai kondisi yang diambil dari stack bernilai false.

Pada intermediate code akhir, label tidak lagi disimpan sebagai nama simbolik seperti L_START atau L_END. Label sudah diselesaikan menjadi nomor instruksi melalui proses backpatching pada code generator. Oleh karena itu, interpreter cukup membaca operand dari instruksi JMP atau JPC sebagai target instruction pointer.

Contoh:

```
JPC 0 12
```

berarti interpreter akan mengambil nilai kondisi dari stack. Jika kondisi false, instruction pointer diubah menjadi 12. Jika kondisi true, eksekusi lanjut ke instruksi berikutnya.

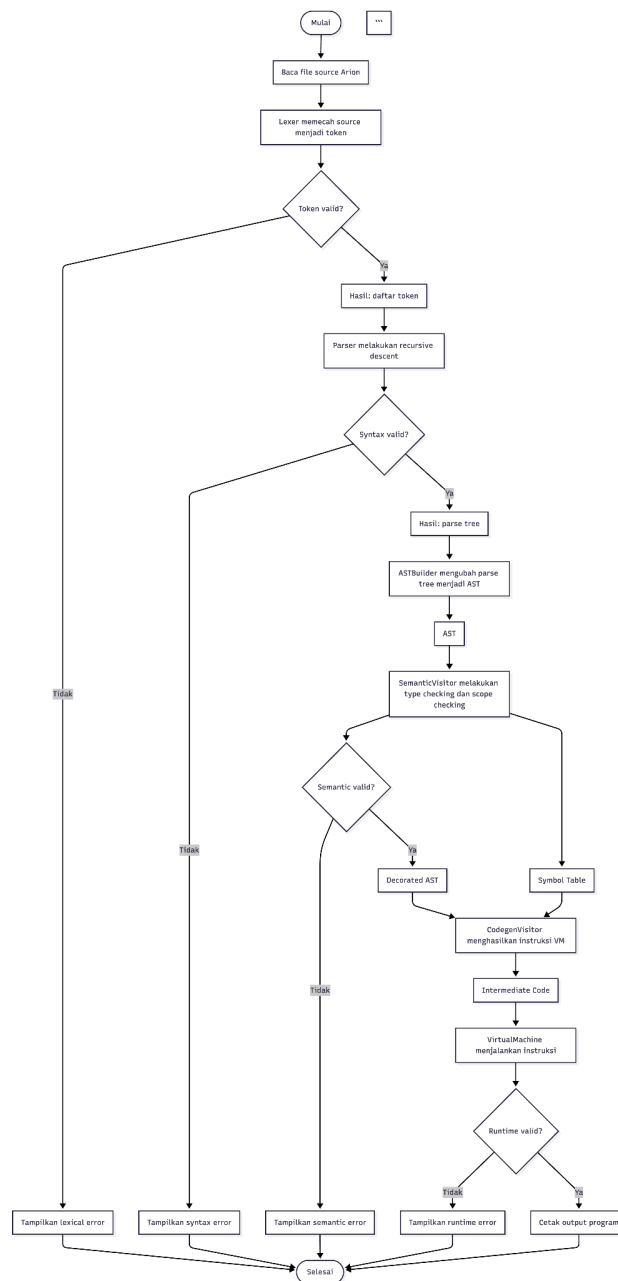
2.6.4 Penanganan Error

Penanganan error dilakukan menggunakan custom exception pada VM. Error yang dapat ditangani antara lain stack underflow, stack overflow, akses memori tidak valid, target jump tidak valid, operasi tidak valid, pembagian dengan nol, dan ketidakcocokan tipe runtime.

Stack underflow terjadi jika interpreter mencoba melakukan pop dari stack kosong. Invalid memory access terjadi jika instruksi mencoba membaca atau menulis alamat di luar batas memori. Division by zero terjadi jika operasi pembagian dilakukan dengan pembagi bernilai nol. Type mismatch terjadi jika operasi tertentu menerima tipe nilai yang tidak sesuai.

Dengan exception khusus, program dapat memberikan pesan error yang lebih informatif dan menghindari crash langsung pada level C++.

2.7 Alur Kerja Program Secara Keseluruhan



Untuk gambar yang lebih jelas dapat dilihat [disini](#)

Alur kerja program dimulai dari pembacaan source code Arion. Source code diproses oleh lexer untuk menghasilkan token. Token kemudian diproses oleh parser untuk membentuk parse tree. Parse tree dikonversi menjadi AST oleh ASTBuilder. Setelah itu, SemanticVisitor melakukan pemeriksaan semantik dan mendekorasi AST dengan informasi tipe, scope, dan referensi symbol table.

Decorated AST dan symbol table kemudian diberikan kepada CodegenVisitor. CodegenVisitor menelusuri AST dan menghasilkan daftar instruksi intermediate code. Instruksi tersebut dicetak

agar dapat dilihat sebagai hasil translasi program. Setelah intermediate code terbentuk, daftar instruksi diberikan kepada VirtualMachine. VirtualMachine menginisialisasi memori, menjalankan instruksi satu per satu, mengelola stack, melakukan operasi OPR, menangani jump, dan menghasilkan output program.

Dengan alur ini, bahasa Arion dapat dijalankan secara penuh sebagai interpreted language. Program tidak dikompilasi menjadi executable native, melainkan menjadi intermediate code yang dieksekusi oleh virtual machine buatan sendiri.

Bab IV

Pengujian

3.1 Metodologi Pengujian

Pengujian dilakukan secara end-to-end, yaitu dengan memberikan source code Arion sebagai input dan memeriksa kebenaran intermediate code serta output program yang dihasilkan. Setiap test case dirancang untuk menguji aspek yang berbeda dari sistem sehingga cakupan pengujian bersifat komprehensif. Verifikasi kebenaran dilakukan dengan membandingkan output yang dihasilkan terhadap hasil perhitungan manual dan memastikan pola instruksi TAC yang dihasilkan sesuai dengan spesifikasi yang diharapkan.

3.2 Test Case 1

Kode Program Input

```
program AssignmentStmt;
var
  x: integer;
begin
  x := 42;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'AssignmentStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    └── [stmt] AssignNode [type:void, lev:1]
        ├── [target] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
        └── [value] NumberNode(value: 42) [type:integer, lev:1]
```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0

12 if	11 RESERVED	VOID	-1	1	0	0
13 mod	12 RESERVED	VOID	-1	1	0	0
14 not	13 RESERVED	VOID	-1	1	0	0
15 of	14 RESERVED	VOID	-1	1	0	0
16 or	15 RESERVED	VOID	-1	1	0	0
17 procedure	16 RESERVED	VOID	-1	1	0	0
18 program	17 RESERVED	VOID	-1	1	0	0
19 record	18 RESERVED	VOID	-1	1	0	0
20 repeat	19 RESERVED	VOID	-1	1	0	0
21 integer	20 TYPE	INTEGER	-1	1	0	1
22 real	21 TYPE	REAL	-1	1	0	1
23 boolean	22 TYPE	BOOLEAN	-1	1	0	1
24 char	23 TYPE	CHAR	-1	1	0	1
25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 AssignmentStmt	37 PROGRAM	VOID	-1	1	0	0
39 x	38 VARIABLE	INTEGER	-1	1	0	0

BTAB

idx	last	lpar	psze	vsze
0	39	-1	0	1
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```
0 INT 0 4
1 LIT 0 42
2 STO 0 3
3 RET 0 0
```

Screenshot

```
ted-ast/dast-tc1-assignment.txt
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 42
2 STO 0 3
3 RET 0 0
```

Penjelasan

Kasus ini memverifikasi unit paling mendasar dari sebuah program, yaitu alokasi memori dan operasi assignment. Saat code generator mengunjungi ProgramNode, ia lebih dulu menghitung ukuran frame program utama. Karena hanya ada satu variabel skalar (x), ukuran frame menjadi VM_FRAME_HEADER_SIZE (3 slot untuk static link, dynamic link, dan return address) ditambah 1 slot variabel, sehingga diterbitkan INT 0 4 pada baris 0. Selanjutnya statement $x := 42$ diproses secara DFS: sisi kanan dievaluasi lebih dulu, yaitu literal 42 menjadi LIT 0 42 (baris 1) yang mendorong nilai ke stack evaluasi. Target assignment berupa variabel sederhana x diterjemahkan menjadi STO 0 3 (baris 2) yang mem-pop puncak stack dan menyimpannya ke slot beralamat 3. Alamat 3 berasal dari adr variabel x (yaitu 0 pada symbol table) ditambah VM_FRAME_HEADER_SIZE, sehingga variabel pengguna pertama selalu menempati slot tepat setelah header. Program ditutup dengan RET 0 0 yang menghentikan VM karena berada pada level terluar.

3.3 Test Case 2

Kode Program Input

```
program IfElseStmt;
var
  x, y: integer;
begin
  if x > 0 then
    y := 1
  else
    y := 2;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'IfElseStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x', 'y') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] IfNode [type:void, lev:1]
    │   ├── [cond] BinOpNode(op: '>') [type:boolean, lev:1]
    │   │   ├── [left] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]
    │   │   └── [right] NumberNode(value: 0) [type:integer, lev:1]
```

```

└─ [then] AssignNode [type:void, lev:1]
└─ [target] VarNode(name: 'y') [tab_index:40,
type:integer, lev:0]
└─ [value] NumberNode(value: 1) [type:integer, lev:1]
└─ [else] AssignNode [type:void, lev:1]
└─ [target] VarNode(name: 'y') [tab_index:40,
type:integer, lev:0]
└─ [value] NumberNode(value: 2) [type:integer, lev:1]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	IfElseStmt	37	PROGRAM	VOID	-1	1	0	0

39	x	38	VARIABLE	INTEGER	-1	1	0	0
40	y	39	VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

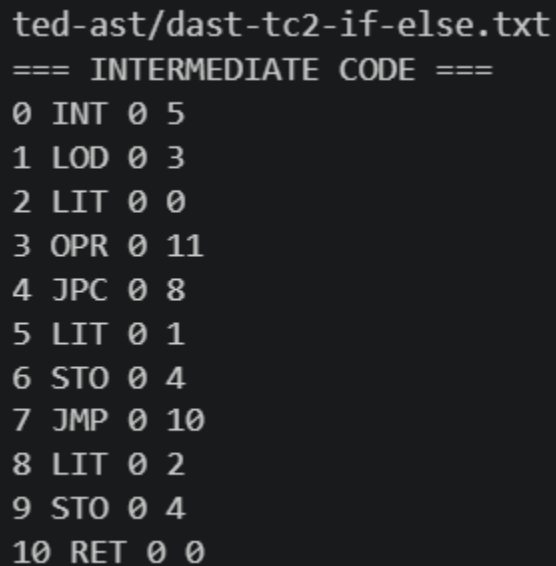
idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```
0 INT 0 5
1 LOD 0 3
2 LIT 0 0
3 OPR 0 11
4 JPC 0 8
5 LIT 0 1
6 STO 0 4
7 JMP 0 10
8 LIT 0 2
9 STO 0 4
10 RET 0 0
```

Screenshot



```
ted-ast/dast-tc2-if-else.txt
=== INTERMEDIATE CODE ===
0 INT 0 5
1 LOD 0 3
2 LIT 0 0
3 OPR 0 11
4 JPC 0 8
5 LIT 0 1
6 STO 0 4
7 JMP 0 10
8 LIT 0 2
9 STO 0 4
10 RET 0 0
```

Penjelasan

Kasus ini menunjukkan bagaimana struktur kontrol bersarang diratakan menjadi barisan linier

menggunakan lompatan dan teknik backpatching. Frame dialokasikan INT 0 5 (3 header + 2 variabel x dan y). Kondisi dibangkitkan lebih dulu: LOD 0 3 memuat x, LIT 0 0 mendorong literal 0, lalu OPR 0 11 (operasi GTR) mem-pop kedua nilai dan mendorong hasil boolean $x > 0$. Berikutnya JPC 0 8 (baris 4): bila kondisi false, eksekusi melompat ke baris 8 (awal blok else); bila true, eksekusi jatuh lurus (fall-through) ke blok then. Saat blok then ($y := 1$) dibangkitkan pada baris 5–6, alamat tujuan JPC belum diketahui, sehingga operannya diisi sementara lalu di-patch menjadi 8 setelah posisi else pasti. Agar setelah then program tidak ikut menjalankan else, diterbitkan JMP 0 10 (baris 7) untuk melompatinya. Blok else ($y := 2$) menempati baris 8–9; kedua cabang menulis ke alamat 4 (variabel y). Inilah pola standar if-else: satu lompatan bersyarat untuk memilih cabang dan satu lompatan mutlak untuk menjaga eksklusivitas keduanya.

3.4 Test Case 3

Kode Program Input
<pre> program WhileStmt; var x: integer; begin x := 0; while x < 3 do begin x := x + 1 end; end. </pre>
Decorated AST input
<pre> === DECORATED AST === ProgramNode(name: 'WhileStmt') [type:void, lev:0] ├─ [decl] VarDeclNode(names: 'x') [type:void, lev:0] │ └─ [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0] └─ [body] BlockNode(btab_index: 1) [type:void, lev:1] ├─ [stmt] AssignNode [type:void, lev:1] │ └─ [target] VarNode(name: 'x') [tab_index:39, type:integer, lev:0] │ └─ [value] NumberNode(value: 0) [type:integer, lev:1] └─ [stmt] WhileNode [type:void, lev:1] ├─ [cond] BinOpNode(op: '<') [type:boolean, lev:1] │ └─ [left] VarNode(name: 'x') [tab_index:39, type:integer, lev:0] │ └─ [right] NumberNode(value: 3) [type:integer, lev:1] └─ [body] BlockNode [type:void, lev:1] └─ [stmt] AssignNode [type:void, lev:1] └─ [target] VarNode(name: 'x') [tab_index:39, type:integer, lev:0] └─ [value] BinOpNode(op: '+') [type:integer, lev:1] └─ [left] VarNode(name: 'x') [tab_index:39, type:integer, lev:0] </pre>

└─ [right] NumberNode(value: 1) [type:integer,

lev:1]

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	WhileStmt	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0

BTAB

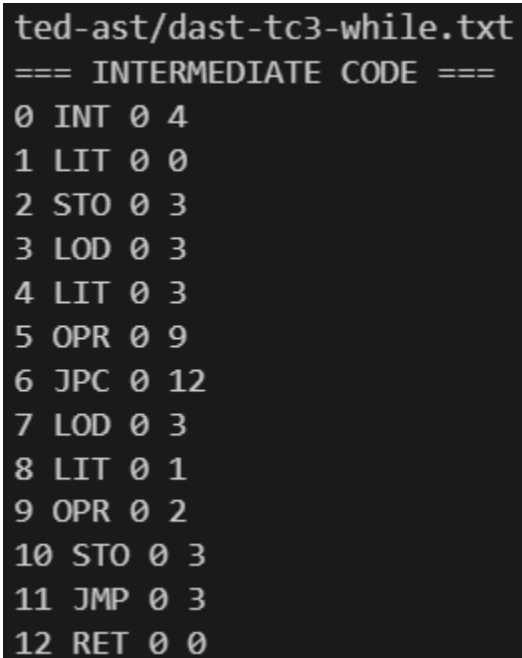
idx	last	lpar	psize	vsze
0	39	-1	0	1
1	-1	-1	0	0

ATAB	idx	xtyp	etyp	eref	low	high	elsz	size
------	-----	------	------	------	-----	------	------	------

Output Program

```
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 0
2 STO 0 3
3 LOD 0 3
4 LIT 0 3
5 OPR 0 9
6 JPC 0 12
7 LOD 0 3
8 LIT 0 1
9 OPR 0 2
10 STO 0 3
11 JMP 0 3
12 RET 0 0
```

Screenshot



```
ted-ast/dast-tc3-while.txt
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 0
2 STO 0 3
3 LOD 0 3
4 LIT 0 3
5 OPR 0 9
6 JPC 0 12
7 LOD 0 3
8 LIT 0 1
9 OPR 0 2
10 STO 0 3
11 JMP 0 3
12 RET 0 0
```

Penjelasan

Kasus ini memverifikasi struktur perulangan dengan menguji kondisi di awal (pre-test loop). Setelah inisialisasi $x := 0$ (baris 1–2), code generator menandai posisi awal kondisi, yaitu baris 3, sebagai titik kembali

(loop entry point). Pada setiap iterasi, kondisi $x < 3$ dibangkitkan melalui instruksi LOD 0 3 untuk memuat nilai x , LIT 0 3 untuk mendorong konstanta 3, dan OPR 0 9 (LSS) untuk menghasilkan nilai boolean hasil perbandingan.

Instruksi JPC 0 12 (baris 6) berfungsi sebagai penjaga gerbang (loop guard). Jika kondisi bernilai false, eksekusi langsung melompat ke baris 12 (RET) sehingga perulangan berakhir. Sebaliknya, jika kondisi bernilai true, badan loop dieksekusi. Pada contoh ini, operasi $x := x + 1$ direalisasikan pada baris 7–10.

Perbedaan utama antara while dan if terletak pada keberadaan instruksi JMP 0 3 (baris 11). Alih-alih melanjutkan eksekusi ke instruksi berikutnya setelah badan program selesai dijalankan, instruksi ini mengembalikan alur eksekusi ke awal kondisi sehingga kondisi dapat dievaluasi kembali. Mekanisme tersebut membentuk siklus uji kondisi ke eksekusi badan lalu uji ulang yang berlangsung hingga nilai x mencapai 3.

Target lompatan pada instruksi JPC ditentukan melalui mekanisme backpatching setelah seluruh badan loop selesai dibangkitkan dan posisi akhir perulangan telah diketahui.

3.5 Test Case 4

Kode Program Input

```
program ForStmt;
var
  i, sum: integer;
begin
  sum := 0;
  for i := 1 to 3 do
  begin
    sum := sum + i
  end;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'ForStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'i', 'sum') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'sum') [tab_index:40, type:integer,
lev:0]
    │       └── [value] NumberNode(value: 0) [type:integer, lev:1]
    └── [stmt] ForNode(var: 'i', dir: to) [type:void, lev:1]
        ├── [from] NumberNode(value: 1) [type:integer, lev:1]
        ├── [to] NumberNode(value: 3) [type:integer, lev:1]
        └── [body] BlockNode [type:void, lev:1]
            └── [stmt] AssignNode [type:void, lev:1]
                └── [target] VarNode(name: 'sum') [tab_index:40,
```



```

type:integer, lev:0]
    └─ [value] BinOpNode(op: '+') [type:integer, lev:1]
      └─ [left] VarNode(name: 'sum') [tab_index:40,
type:integer, lev:0]
    └─ [right] VarNode(name: 'i') [tab_index:39,
type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	ForStmt	37	PROGRAM	VOID	-1	1	0	0
39	i	38	VARIABLE	INTEGER	-1	1	0	0
40	sum	39	VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```
0 INT 0 5
1 LIT 0 0
2 STO 0 4
3 LIT 0 1
4 STO 0 3
5 LOD 0 3
6 LIT 0 3
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 RET 0 0
```

Screenshot

```
ted-ast/dast-tc4-for-to.txt
```

```
=== INTERMEDIATE CODE ===
```

```
0 INT 0 5
1 LIT 0 0
2 STO 0 4
3 LIT 0 1
4 STO 0 3
5 LOD 0 3
6 LIT 0 3
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 RET 0 0
```

Penjelasan

Kasus ini memperlihatkan penerjemahan struktur for yang lebih kompleks dibandingkan while karena melibatkan tiga komponen utama: inisialisasi, pengujian batas, dan pembaruan otomatis variabel kendali. Pada contoh ini, variabel *i* disimpan pada alamat 3 dan sum pada alamat 4.

Setelah inisialisasi $\text{sum} := 0$ (baris 1–2), loop diinisialisasi dengan $i := 1$ (baris 3–4). Baris 5 kemudian ditandai sebagai titik awal pengujian kondisi. Instruksi LOD 0 3 memuat nilai *i*, LIT 0 3 mendorong nilai batas atas, dan OPR 0 12 (LEQ) mengevaluasi kondisi $i \leq 3$. Operator $\leq$ digunakan karena perulangan bertipe to; untuk perulangan downto, code generator akan menggunakan operator $\geq$ (GEQ).

Instruksi JPC 0 18 berfungsi untuk mengakhiri loop apabila kondisi tidak lagi terpenuhi, yaitu ketika nilai *i* telah melampaui batas atas. Jika kondisi masih bernilai true, badan loop dieksekusi. Pada contoh ini, operasi $\text{sum} := \text{sum} + i$ direalisasikan pada baris 9–12.

Bagian yang membedakan for dari while terlihat pada baris 13–16. Code generator secara otomatis menyisipkan instruksi pembaruan variabel kendali berupa $i := i + 1$, meskipun operasi tersebut tidak ditulis secara eksplisit oleh pengguna. Setelah proses increment selesai, instruksi JMP 0 5 mengembalikan alur eksekusi ke titik pengujian batas untuk memulai iterasi berikutnya.

Jika program dijalankan, perulangan ini akan menghitung penjumlahan berurutan $1 + 2 + 3$ dan menyimpan

hasilnya ke dalam variabel sum.

3.6 Test Case 5

Kode Program Input

```
program RepeatUntilStmt;
var
  x: integer;
begin
  x := 3;
  repeat
    x := x - 1
  until x == 0;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'RepeatUntilStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
    │       └── [value] NumberNode(value: 3) [type:integer, lev:1]
    └── [stmt] RepeatNode [type:void, lev:1]
        ├── [stmt] AssignNode [type:void, lev:1]
        │   └── [target] VarNode(name: 'x') [tab_index:39,
type:integer, lev:0]
        │       ├── [value] BinOpNode(op: '-') [type:integer, lev:1]
        │       │   ├── [left] VarNode(name: 'x') [tab_index:39,
type:integer, lev:0]
        │       │   └── [right] NumberNode(value: 1) [type:integer, lev:1]
        │       └── [until] BinOpNode(op: '==') [type:boolean, lev:1]
        │           ├── [left] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
        │           └── [right] NumberNode(value: 0) [type:integer, lev:1]
```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0

4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	RepeatUntilStmt	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0

BTAB

idx	last	lpar	psze	vsze
0	39	-1	0	1
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

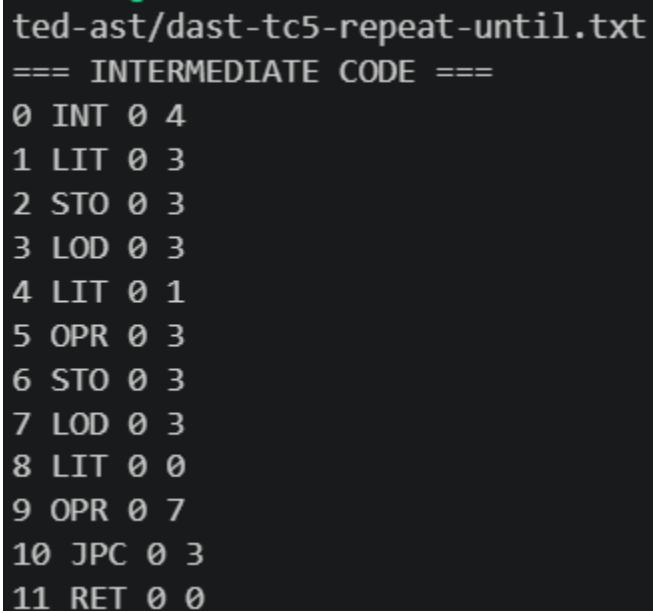
0 INT 0 4

```

1 LIT 0 3
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 3
6 STO 0 3
7 LOD 0 3
8 LIT 0 0
9 OPR 0 7
10 JPC 0 3
11 RET 0 0

```

Screenshot



```

ted-ast/dast-tc5-repeat-until.txt
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 3
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 3
6 STO 0 3
7 LOD 0 3
8 LIT 0 0
9 OPR 0 7
10 JPC 0 3
11 RET 0 0

```

Penjelasan

Kasus ini memverifikasi struktur perulangan repeat-until, yaitu perulangan dengan pengujian kondisi di akhir (post-test loop), kebalikan dari while. Karena kondisi diperiksa setelah badan loop dieksekusi, badan perulangan dijamin berjalan minimal satu kali.

Setelah inisialisasi $x := 3$ (baris 1–2), baris 3 ditandai sebagai awal loop. Berbeda dengan while, badan perulangan dibangkitkan terlebih dahulu. Pada contoh ini, operasi $x := x - 1$ direalisasikan pada baris 3–6 menggunakan instruksi OPR 0 3 (SUB).

Setelah badan loop selesai dieksekusi, kondisi $x == 0$ dibangkitkan pada baris 7–9. Instruksi LOD 0 3 memuat nilai x , LIT 0 0 mendorong konstanta 0, dan OPR 0 7 (EQL) melakukan pengujian kesamaan.

Karakteristik utama pola repeat-until terlihat pada instruksi JPC 0 3 (baris 10). Karena JPC melakukan lompatan ketika kondisi bernilai false, eksekusi akan kembali ke awal loop selama kondisi $x == 0$ belum terpenuhi. Ketika kondisi akhirnya bernilai true, lompatan tidak dilakukan dan eksekusi berlanjut ke instruksi

berikutnya di luar perulangan.

Dengan demikian, alur eksekusi yang terbentuk adalah badan ke uji kondisi lalu ulangi bila kondisi masih false. Tidak diperlukan lompatan tambahan untuk memasuki loop karena badan perulangan selalu dieksekusi terlebih dahulu sebelum kondisi diperiksa.

3.7 Test Case 6

Kode Program Input

```
program CaseStmt;
var
  choice, result: integer;
begin
  choice := 2;
  case choice of
    1: result := 10;
    2: result := 20
  end;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'CaseStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'choice', 'result') [type:void, lev:0]
│   ├── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
│   └── [body] BlockNode(btab_index: 1) [type:void, lev:1]
│       ├── [stmt] AssignNode [type:void, lev:1]
│       │   ├── [target] VarNode(name: 'choice') [tab_index:39, type:integer, lev:0]
│       │   └── [value] NumberNode(value: 2) [type:integer, lev:1]
│       └── [stmt] CaseNode [type:void, lev:1]
│           ├── [selector] VarNode(name: 'choice') [tab_index:39, type:integer, lev:0]
│           ├── [label] NumberNode(value: 1) [type:integer, lev:1]
│           │   └── [stmt] AssignNode [type:void, lev:1]
│           │       ├── [target] VarNode(name: 'result') [tab_index:40, type:integer, lev:0]
│           │       └── [value] NumberNode(value: 10) [type:integer, lev:1]
│           ├── [label] NumberNode(value: 2) [type:integer, lev:1]
│           │   └── [stmt] AssignNode [type:void, lev:1]
│           │       ├── [target] VarNode(name: 'result') [tab_index:40, type:integer, lev:0]
│           │       └── [value] NumberNode(value: 20) [type:integer, lev:1]
└── [value] NumberNode(value: 20) [type:integer, lev:1]

=== SYMBOL TABLE ===
TAB
```

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	CaseStmt	37	PROGRAM	VOID	-1	1	0	0
39	choice	38	VARIABLE	INTEGER	-1	1	0	0
40	result	39	VARIABLE	INTEGER	-1	1	0	1

BTAB

idx	last	lpar	psze	vsze
0	40	-1	0	2
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program
<pre>=== INTERMEDIATE CODE === 0 INT 0 5 1 LIT 0 2 2 STO 0 3 3 LOD 0 3 4 LIT 0 1 5 OPR 0 7 6 JPC 0 10 7 LIT 0 10 8 STO 0 4 9 JMP 0 16 10 LOD 0 3 11 LIT 0 2 12 OPR 0 7 13 JPC 0 16 14 LIT 0 20 15 STO 0 4 16 RET 0 0</pre>
Screenshot

```
ted-ast/dast-tc6-case.txt
```

```
=== INTERMEDIATE CODE ===
```

```
0 INT 0 5
1 LIT 0 2
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 7
6 JPC 0 10
7 LIT 0 10
8 STO 0 4
9 JMP 0 16
10 LOD 0 3
11 LIT 0 2
12 OPR 0 7
13 JPC 0 16
14 LIT 0 20
15 STO 0 4
16 RET 0 0
```

Penjelasan

Kasus ini menunjukkan penerjemahan struktur case sebagai rangkaian perbandingan berurutan (cascade of comparisons). Setelah inisialisasi choice := 2 (baris 1–2), mesin virtual memproses setiap cabang secara berurutan hingga menemukan label yang sesuai.

Pada cabang pertama (label 1), nilai selektor dievaluasi dengan memuat choice menggunakan LOD 0 3, mendorong konstanta 1 melalui LIT 0 1, lalu membandingkannya menggunakan OPR 0 7 (EQL). Hasil perbandingan berupa nilai boolean di puncak stack. Jika hasilnya false, instruksi JPC 0 10 mengalihkan eksekusi ke cabang berikutnya. Sebaliknya, jika hasilnya true, badan cabang result := 10 (baris 7–8) dieksekusi, kemudian instruksi JMP 0 16 digunakan untuk langsung keluar dari struktur case sehingga cabang-cabang berikutnya tidak ikut dievaluasi.

Hal yang penting untuk diperhatikan adalah nilai selektor harus dimuat ulang pada setiap cabang. Oleh karena itu, LOD 0 3 muncul kembali pada baris 10 untuk cabang kedua. Langkah ini diperlukan karena operasi perbandingan mengonsumsi operand yang digunakan, sehingga nilai selektor tidak lagi tersedia di stack setelah pengujian sebelumnya selesai.

Cabang kedua (label 2) mengikuti pola yang sama: nilai selektor dimuat, dibandingkan dengan label cabang, lalu badan cabang dieksekusi apabila kondisi terpenuhi. Namun, karena cabang ini merupakan cabang terakhir, tidak diperlukan instruksi JMP tambahan setelah badan cabang. Jika label cocok, eksekusi dapat langsung berlanjut ke instruksi berikutnya setelah struktur case, yaitu pada baris 16.

Seluruh instruksi JMP yang digunakan untuk keluar dari cabang-cabang yang berhasil dicocokkan akan di-patch menuju alamat akhir struktur case setelah seluruh cabang selesai dibangkitkan. Dengan mekanisme ini, hanya satu cabang yang dapat dieksekusi dan evaluasi berhenti segera setelah ditemukan kecocokan pertama.

3.8 Test Case 7

Kode Program Input

```
program WritelnStmt;
var
  x: integer;
begin
  x := 7;
  writeln(x);
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'WritelnStmt') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
    │       └── [value] NumberNode(value: 7) [type:integer, lev:1]
    └── [stmt] ProcCallNode(name: 'writeln') [tab_index:34, type:void,
lev:1]
        └── [arg] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0

12 if	11 RESERVED	VOID	-1	1	0	0
13 mod	12 RESERVED	VOID	-1	1	0	0
14 not	13 RESERVED	VOID	-1	1	0	0
15 of	14 RESERVED	VOID	-1	1	0	0
16 or	15 RESERVED	VOID	-1	1	0	0
17 procedure	16 RESERVED	VOID	-1	1	0	0
18 program	17 RESERVED	VOID	-1	1	0	0
19 record	18 RESERVED	VOID	-1	1	0	0
20 repeat	19 RESERVED	VOID	-1	1	0	0
21 integer	20 TYPE	INTEGER	-1	1	0	1
22 real	21 TYPE	REAL	-1	1	0	1
23 boolean	22 TYPE	BOOLEAN	-1	1	0	1
24 char	23 TYPE	CHAR	-1	1	0	1
25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 WritelnStmt	37 PROGRAM	VOID	-1	1	0	0
39 x	38 VARIABLE	INTEGER	-1	1	0	0

BTAB

idx	last	lpar	psize	vsze
0	39	-1	0	1
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```

0 INT 0 4
1 LIT 0 7
2 STO 0 3
3 LOD 0 3
4 OPR 0 14
5 RET 0 0

```

Screenshot

```

ted-ast/dast-tc7-writeln.txt
=== INTERMEDIATE CODE ===
0 INT 0 4
1 LIT 0 7
2 STO 0 3
3 LOD 0 3
4 OPR 0 14
5 RET 0 0

```

Penjelasan

Kasus ini memverifikasi prosedur bawaan `writeln`, yang merupakan jalur utama keluaran (output) program pada mesin virtual. Setelah inisialisasi $x := 7$ (baris 1–2), pemanggilan `writeln(x)` diterjemahkan menjadi dua tahap: pertama, argumen x dievaluasi melalui instruksi `LOD 0 3` yang memuat nilai x ke stack; kedua, pemanggilan prosedur `writeln` dipetakan menjadi instruksi `OPR 0 14` (WRTLN).

Saat dieksekusi, WRTLN mengambil (pop) nilai pada puncak stack, mengubahnya ke representasi teks, mencetaknya ke keluaran standar, kemudian menambahkan karakter newline. Dengan demikian, nilai yang sebelumnya berada di stack akan ditampilkan sebagai output program dan tidak lagi tersimpan pada stack setelah proses pencetakan selesai.

Untuk pemanggilan `writeln` dengan lebih dari satu argumen, code generator menghasilkan serangkaian instruksi keluaran sesuai urutan argumen. Argumen-argumen selain yang terakhir dicetak menggunakan `OPR 0 13` (WRT), yang menampilkan nilai tanpa menambahkan newline. Sementara itu, argumen terakhir dicetak menggunakan `OPR 0 14` (WRTLN) sehingga keluaran diakhiri dengan perpindahan baris.

Mekanisme ini sesuai dengan spesifikasi yang menyatakan bahwa `writeln` dapat menerima banyak argumen. Kombinasi instruksi WRT dan WRTLN memastikan seluruh argumen dicetak secara berurutan, dengan hanya satu karakter newline yang ditambahkan setelah argumen terakhir.

3.9 Test Case 8

Kode Program Input

```

program LiteralExpr;
var
  i: integer;
  r: real;
  c: char;
  s: string;
  b: boolean;
begin
  i := 42;
  r := 3.14;
  c := 'A';
  s := 'Hello';

```

```

b := true;
end.

```

Decorated AST input

=== DECORATED AST ===

```

ProgramNode(name: 'LiteralExpr') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'i') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
├── [decl] VarDeclNode(names: 'r') [type:void, lev:0]
│   └── [type] VarNode(name: 'real') [tab_index:22, type:real, lev:0]
├── [decl] VarDeclNode(names: 'c') [type:void, lev:0]
│   └── [type] VarNode(name: 'char') [tab_index:24, type:char, lev:0]
├── [decl] VarDeclNode(names: 's') [type:void, lev:0]
│   └── [type] VarNode(name: 'string') [tab_index:25, type:string,
lev:0]
├── [decl] VarDeclNode(names: 'b') [type:void, lev:0]
│   └── [type] VarNode(name: 'boolean') [tab_index:23, type:boolean,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'i') [tab_index:39, type:integer,
lev:0]
    │       └── [value] NumberNode(value: 42) [type:integer, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'r') [tab_index:40, type:real,
lev:0]
    │       └── [value] RealNode(value: 3.14) [type:real, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'c') [tab_index:41, type:char,
lev:0]
    │       └── [value] CharNode(value: 'A') [type:char, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 's') [tab_index:42, type:string,
lev:0]
    │       └── [value] StringNode(value: "Hello") [type:string, lev:1]
    └── [stmt] AssignNode [type:void, lev:1]
        └── [target] VarNode(name: 'b') [tab_index:43, type:boolean,
lev:0]
            └── [value] BoolNode(value: true) [type:boolean, lev:1]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0

6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	LiteralExpr	37	PROGRAM	VOID	-1	1	0	0
39	i	38	VARIABLE	INTEGER	-1	1	0	0
40	r	39	VARIABLE	REAL	-1	1	0	1
41	c	40	VARIABLE	CHAR	-1	1	0	2
42	s	41	VARIABLE	STRING	-1	1	0	3
43	b	42	VARIABLE	BOOLEAN	-1	1	0	4

BTAB

idx	last	lpar	psze	vsze
0	43	-1	0	5
1	-1	-1	0	0

ATAB

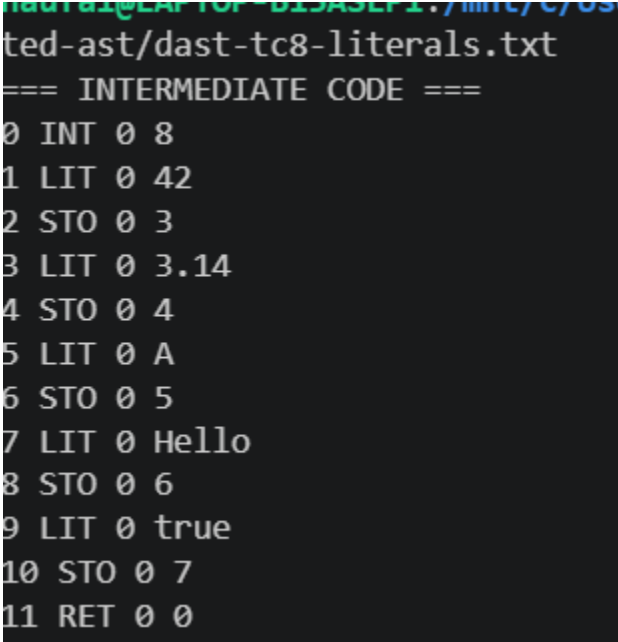
idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

```
=== INTERMEDIATE CODE ===
```

```
0 INT 0 8
1 LIT 0 42
2 STO 0 3
3 LIT 0 3.14
4 STO 0 4
5 LIT 0 A
6 STO 0 5
7 LIT 0 Hello
8 STO 0 6
9 LIT 0 true
10 STO 0 7
11 RET 0 0
```

Screenshot



```
ted-ast/dast-tc8-literals.txt
=== INTERMEDIATE CODE ===
0 INT 0 8
1 LIT 0 42
2 STO 0 3
3 LIT 0 3.14
4 STO 0 4
5 LIT 0 A
6 STO 0 5
7 LIT 0 Hello
8 STO 0 6
9 LIT 0 true
10 STO 0 7
11 RET 0 0
```

Penjelasan

Kasus ini memverifikasi bahwa instruksi LIT mampu memuat seluruh lima tipe primitif dalam bahasa Arion, tidak hanya bilangan bulat. Pada contoh ini, lima variabel dideklarasikan sehingga activation record memiliki ukuran INT 0 8, yang terdiri atas tiga sel header dan lima variabel yang menempati alamat 3 hingga 7.

Setiap operasi penugasan (assignment) diterjemahkan menjadi pasangan instruksi LIT–STO. Nilai literal yang digunakan mencakup seluruh tipe primitif yang didukung bahasa, yaitu integer (42), real (3.14), char ('A'), string ("Hello"), dan boolean (true). Instruksi LIT bertugas mendorong nilai literal ke stack, sedangkan STO menyimpannya ke alamat variabel yang sesuai.

Kemampuan ini dimungkinkan karena setiap nilai direpresentasikan menggunakan objek VMValue, yang berfungsi sebagai tagged union. Selain menyimpan data aktual, setiap objek juga membawa informasi tipe yang menyertainya. Dengan demikian, mesin virtual dapat membedakan apakah suatu nilai merupakan

integer, real, char, string, atau boolean, meskipun seluruhnya diproses melalui mekanisme stack yang sama.

Ketika nilai ditampilkan sebagai keluaran program, representasi teksnya dihasilkan oleh metode `to_output_string`, yang menyesuaikan format keluaran berdasarkan tipe nilai yang disimpan. Sebagai contoh, nilai numerik dicetak sebagai angka, nilai karakter sebagai satu karakter, nilai string sebagai rangkaian karakter, dan nilai boolean sebagai representasi logisnya.

Dukungan terhadap berbagai tipe data pada instruksi LIT menjadi fondasi bagi mekanisme runtime type checking. Karena setiap nilai selalu membawa informasi tipenya, mesin virtual dapat mengetahui tipe sebenarnya dari setiap data yang berada di memori maupun stack, sehingga operasi-operasi yang dijalankan dapat divalidasi dan diproses secara aman sesuai aturan sistem tipe bahasa Arion.

3.10 Test Case 9

Kode Program Input	
<pre>program BinaryExpr; var x, y, z: integer; begin z := x + y * 2; end.</pre>	
Decorated AST input	
<pre>=== DECORATED AST === ProgramNode(name: 'BinaryExpr') [type:void, lev:0] ├─ [decl] VarDeclNode(names: 'x', 'y', 'z') [type:void, lev:0] │ └─ [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0] └─ [body] BlockNode(btab_index: 1) [type:void, lev:1] └─ [stmt] AssignNode [type:void, lev:1] └─ [target] VarNode(name: 'z') [tab_index:41, type:integer, lev:0] └─ [value] BinOpNode(op: '+') [type:integer, lev:1] └─ [left] VarNode(name: 'x') [tab_index:39, type:integer, lev:0] └─ [right] BinOpNode(op: '*') [type:integer, lev:1] └─ [left] VarNode(name: 'y') [tab_index:40, type:integer, lev:0] └─ [right] NumberNode(value: 2) [type:integer, lev:1] === SYMBOL TABLE === TAB idx id link obj type ref nrm lev adr 0 and -1 RESERVED VOID -1 1 0 0 1 array 0 RESERVED VOID -1 1 0 0 2 begin 1 RESERVED VOID -1 1 0 0 3 case 2 RESERVED VOID -1 1 0 0</pre>	

4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	BinaryExpr	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0
40	y	39	VARIABLE	INTEGER	-1	1	0	1
41	z	40	VARIABLE	INTEGER	-1	1	0	2

BTAB

idx	last	lpar	psze	vsze
0	41	-1	0	3
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

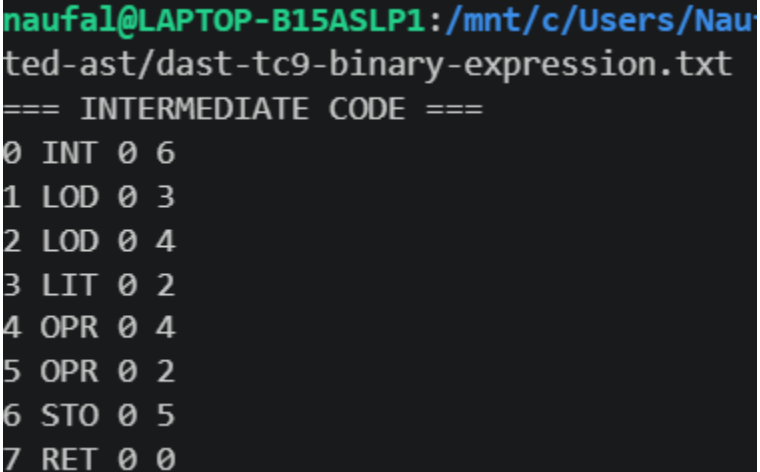
Output Program

```

=== INTERMEDIATE CODE ===
0 INT 0 6
1 LOD 0 3
2 LOD 0 4
3 LIT 0 2
4 OPR 0 4
5 OPR 0 2
6 STO 0 5
7 RET 0 0

```

Screenshot



```

naufal@LAPTOP-B15ASLP1:/mnt/c/Users/Nau
ted-ast/dast-tc9-binary-expression.txt
=== INTERMEDIATE CODE ===
0 INT 0 6
1 LOD 0 3
2 LOD 0 4
3 LIT 0 2
4 OPR 0 4
5 OPR 0 2
6 STO 0 5
7 RET 0 0

```

Penjelasan

Kasus ini membuktikan bahwa penelusuran Abstract Syntax Tree (AST) secara DFS secara otomatis menghormati presedensi operator tanpa memerlukan logika tambahan pada code generator. Ekspresi $x + y * 2$ telah diparse menjadi pohon dengan $+$ sebagai akar dan $y * 2$ sebagai sub-pohon kanan karena operator $*$ memiliki presedensi lebih tinggi.

Saat membangkitkan kode, code generator terlebih dahulu memuat x (LOD 0 3), kemudian mengevaluasi sub-ekspresi $y * 2$ dengan membangkitkan y (LOD 0 4), 2 (LIT 0 2), dan OPR 0 4 (MUL). Hasil perkalian dibiarkan di puncak stack, lalu OPR 0 2 (ADD) menjumlahkannya dengan x .

Pola operan ke operan lalu operasi merupakan karakteristik stack machine, di mana setiap operasi selalu menemukan operand yang dibutuhkan di puncak stack. Hasil akhir ekspresi kemudian disimpan ke variabel z menggunakan STO 0 5.

3.11 Test Case 10

Kode Program Input

```

program UnaryBooleanExpr;
var

```

```

x: integer;
flag, result: boolean;
begin
  result := not flag or (x > 5);
  x := -x;
end.

```

Decorated AST input

```

=== DECORATED AST ===
ProgramNode(name: 'UnaryBooleanExpr') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0]
├── [decl] VarDeclNode(names: 'flag', 'result') [type:void, lev:0]
│   └── [type] VarNode(name: 'boolean') [tab_index:23, type:boolean, lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] VarNode(name: 'result') [tab_index:41, type:boolean, lev:0]
    │   └── [value] BinOpNode(op: 'or') [type:boolean, lev:1]
    │       ├── [left] UnaryOpNode(op: 'not') [type:boolean, lev:1]
    │       │   └── [operand] VarNode(name: 'flag') [tab_index:40, type:boolean, lev:0]
    │       └── [right] BinOpNode(op: '>') [type:boolean, lev:1]
    │           ├── [left] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]
    │           └── [right] NumberNode(value: 5) [type:integer, lev:1]
    └── [stmt] AssignNode [type:void, lev:1]
        ├── [target] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]
        └── [value] UnaryOpNode(op: '-') [type:integer, lev:1]
            └── [operand] VarNode(name: 'x') [tab_index:39, type:integer, lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0

12 if	11 RESERVED	VOID	-1	1	0	0
13 mod	12 RESERVED	VOID	-1	1	0	0
14 not	13 RESERVED	VOID	-1	1	0	0
15 of	14 RESERVED	VOID	-1	1	0	0
16 or	15 RESERVED	VOID	-1	1	0	0
17 procedure	16 RESERVED	VOID	-1	1	0	0
18 program	17 RESERVED	VOID	-1	1	0	0
19 record	18 RESERVED	VOID	-1	1	0	0
20 repeat	19 RESERVED	VOID	-1	1	0	0
21 integer	20 TYPE	INTEGER	-1	1	0	1
22 real	21 TYPE	REAL	-1	1	0	1
23 boolean	22 TYPE	BOOLEAN	-1	1	0	1
24 char	23 TYPE	CHAR	-1	1	0	1
25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 UnaryBooleanExpr	37 PROGRAM	VOID	-1	1	0	0
39 x	38 VARIABLE	INTEGER	-1	1	0	0
40 flag	39 VARIABLE	BOOLEAN	-1	1	0	1
41 result	40 VARIABLE	BOOLEAN	-1	1	0	2

BTAB

idx	last	lpar	psze	vsze
0	41	-1	0	3
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```

0 INT 0 6
1 LOD 0 4
2 LIT 0 false
3 OPR 0 7
4 LOD 0 3
5 LIT 0 5
6 OPR 0 11
7 OPR 0 16

```

```
8 STO 0 5
9 LOD 0 3
10 OPR 0 1
11 STO 0 3
12 RET 0 0
```

Screenshot

```
=== INTERMEDIATE CODE ===
0 INT 0 6
1 LOD 0 4
2 LIT 0 false
3 OPR 0 7
4 LOD 0 3
5 LIT 0 5
6 OPR 0 11
7 OPR 0 16
8 STO 0 5
9 LOD 0 3
10 OPR 0 1
11 STO 0 3
12 RET 0 0
```

Penjelasan

Kasus ini menguji operator unari not dan -, serta operator boolean or. Operator not diterjemahkan tanpa instruksi khusus, melainkan sebagai perbandingan flag == false, yaitu melalui LOD 0 4, LIT 0 false, dan OPR 0 7 (EQL). Pendekatan ini memanfaatkan operasi yang sudah tersedia di mesin virtual.

Selanjutnya, sub-ekspresi $x > 5$ dibangkitkan menggunakan LOD, LIT, dan OPR 0 11 (GTR). Hasil kedua ekspresi boolean kemudian digabungkan dengan OPR 0 16 (OR) dan disimpan ke variabel result.

Pernyataan $x := -x$ memperlihatkan penggunaan operator unari minus. Nilai x dimuat ke stack melalui LOD 0 3, kemudian dinegasikan menggunakan OPR 0 1 (NEG), yang bekerja langsung pada nilai di puncak stack tanpa memerlukan operand kedua. Hasilnya kemudian disimpan kembali ke x.

Kasus ini juga menegaskan bahwa operator logika and dan or dipetakan ke opcode khusus (15 dan 16) yang diimplementasikan oleh mesin virtual.

3.12 Test Case 11

Kode Program Input

```

program OperatorMappingExpr;
var
  a, b, q, m: integer;
  r: real;
  eq, ne, le: boolean;
begin
  q := a div b;
  m := a mod b;
  r := a / b;
  eq := a == b;
  ne := a <> b;
  le := a <= b;
end.

```

Decorated AST input

```

=== DECORATED AST ===
ProgramNode(name: 'OperatorMappingExpr') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'a', 'b', 'q', 'm') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
├── [decl] VarDeclNode(names: 'r') [type:void, lev:0]
│   └── [type] VarNode(name: 'real') [tab_index:22, type:real, lev:0]
├── [decl] VarDeclNode(names: 'eq', 'ne', 'le') [type:void, lev:0]
│   └── [type] VarNode(name: 'boolean') [tab_index:23, type:boolean,
lev:0]
└── [body] BlockNode(btab_index: 1) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'q') [tab_index:41, type:integer,
lev:0]
    │       └── [value] BinOpNode(op: 'div') [type:integer, lev:1]
    │           ├── [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
    │           └── [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'm') [tab_index:42, type:integer,
lev:0]
    │       └── [value] BinOpNode(op: 'mod') [type:integer, lev:1]
    │           ├── [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
    │           └── [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   └── [target] VarNode(name: 'r') [tab_index:43, type:real,
lev:0]
    │       └── [value] BinOpNode(op: '/') [type:real, lev:1]
    │           ├── [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
    │           └── [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]

```

```

└─ [stmt] AssignNode [type:void, lev:1]
└─ [target] VarNode(name: 'eq') [tab_index:44, type:boolean,
lev:0]
└─ [value] BinOpNode(op: '==') [type:boolean, lev:1]
└─ [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
└─ [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]
└─ [stmt] AssignNode [type:void, lev:1]
└─ [target] VarNode(name: 'ne') [tab_index:45, type:boolean,
lev:0]
└─ [value] BinOpNode(op: '<>') [type:boolean, lev:1]
└─ [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
└─ [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]
└─ [stmt] AssignNode [type:void, lev:1]
└─ [target] VarNode(name: 'le') [tab_index:46, type:boolean,
lev:0]
└─ [value] BinOpNode(op: '<=') [type:boolean, lev:1]
└─ [left] VarNode(name: 'a') [tab_index:39, type:integer,
lev:0]
└─ [right] VarNode(name: 'b') [tab_index:40, type:integer,
lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1

23 boolean	22 TYPE	BOOLEAN	-1	1	0	1
24 char	23 TYPE	CHAR	-1	1	0	1
25 string	24 TYPE	STRING	-1	1	0	1
26 then	25 RESERVED	VOID	-1	1	0	0
27 to	26 RESERVED	VOID	-1	1	0	0
28 type	27 RESERVED	VOID	-1	1	0	0
29 until	28 RESERVED	VOID	-1	1	0	0
30 var	29 RESERVED	VOID	-1	1	0	0
31 while	30 RESERVED	VOID	-1	1	0	0
32 true	31 CONSTANT	BOOLEAN	-1	1	0	1
33 false	32 CONSTANT	BOOLEAN	-1	1	0	0
34 writeln	33 PROCEDURE	VOID	-1	1	0	0
35 readln	34 PROCEDURE	VOID	-1	1	0	0
36 write	35 PROCEDURE	VOID	-1	1	0	0
37 read	36 PROCEDURE	VOID	-1	1	0	0
38 OperatorMappingExpr	37 PROGRAM	VOID	-1	1	0	0
39 a	38 VARIABLE	INTEGER	-1	1	0	0
40 b	39 VARIABLE	INTEGER	-1	1	0	1
41 q	40 VARIABLE	INTEGER	-1	1	0	2
42 m	41 VARIABLE	INTEGER	-1	1	0	3
43 r	42 VARIABLE	REAL	-1	1	0	4
44 eq	43 VARIABLE	BOOLEAN	-1	1	0	5
45 ne	44 VARIABLE	BOOLEAN	-1	1	0	6
46 le	45 VARIABLE	BOOLEAN	-1	1	0	7

BTAB

idx	last	lpar	psze	vsze
0	46	-1	0	8
1	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```

0 INT 0 11
1 LOD 0 3
2 LOD 0 4
3 OPR 0 17
4 STO 0 5
5 LOD 0 3
6 LOD 0 4
7 OPR 0 6
8 STO 0 6
9 LOD 0 3
10 LOD 0 4
11 OPR 0 5
12 STO 0 7
13 LOD 0 3

```

```
14 LOD 0 4
15 OPR 0 7
16 STO 0 8
17 LOD 0 3
18 LOD 0 4
19 OPR 0 8
20 STO 0 9
21 LOD 0 3
22 LOD 0 4
23 OPR 0 12
24 STO 0 10
25 RET 0 0
```

Screenshot

```
ted-ast/dast-tc11-operator-mapping.txt
=== INTERMEDIATE CODE ===
0 INT 0 11
1 LOD 0 3
2 LOD 0 4
3 OPR 0 17
4 STO 0 5
5 LOD 0 3
6 LOD 0 4
7 OPR 0 6
8 STO 0 6
9 LOD 0 3
10 LOD 0 4
11 OPR 0 5
12 STO 0 7
13 LOD 0 3
14 LOD 0 4
15 OPR 0 7
16 STO 0 8
17 LOD 0 3
18 LOD 0 4
19 OPR 0 8
20 STO 0 9
21 LOD 0 3
22 LOD 0 4
23 OPR 0 12
24 STO 0 10
```

Penjelasan
Kasus ini memvalidasi mapper yang menerjemahkan operator pada AST menjadi OprCode yang sesuai. Delapan variabel menghasilkan frame berukuran INT 0 11, sementara enam operasi penugasan mengikuti pola yang sama: memuat operand a dan b, menjalankan satu instruksi OPR, lalu menyimpan hasilnya. Perbedaan setiap operasi terletak pada kode operasinya, yaitu div -> OPR 0 17 (IDIV), mod -> OPR 0 6 (MOD), / -> OPR 0 5 (DIV), == -> OPR 0 7 (EQL), <> -> OPR 0 8 (NEQ), dan <= -> OPR 0 12 (LEQ). Perbedaan antara div (IDIV) dan / (DIV) menjadi aspek penting pada pengujian ini. Keduanya merupakan operasi pembagian, tetapi div bekerja pada bilangan bulat dan menghasilkan hasil bertipe bulat, sedangkan / menghasilkan nilai bertipe real. Dengan demikian, kasus ini menegaskan kelengkapan dan ketepatan pemetaan operator oleh code generator sebelum instruksi dieksekusi oleh mesin virtual.

3.13 Test Case 12

Kode Program Input
<pre> program ArrayAccessExpr; var i, value: integer; arr: array [1..5] of integer; begin value := arr[i]; end. </pre>
Decorated AST input
<pre> === DECORATED AST === ProgramNode(name: 'ArrayAccessExpr') [type:void, lev:0] ├── [decl] VarDeclNode(names: 'i', 'value') [type:void, lev:0] │ └── [type] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0] ├── [decl] VarDeclNode(names: 'arr') [type:void, lev:0] │ └── [type] ArrayTypeNode [tab_index:0, type:array, lev:0] │ ├── [index] SubrangeNode [type:integer, lev:0] │ │ ├── [low] NumberNode(value: 1) [type:integer, lev:0] │ │ └── [high] NumberNode(value: 5) [type:integer, lev:0] │ └── [elem] VarNode(name: 'integer') [tab_index:21, type:integer, lev:0] └── [body] BlockNode(btab_index: 1) [type:void, lev:1] └── [stmt] AssignNode [type:void, lev:1] ├── [target] VarNode(name: 'value') [tab_index:40, type:integer, lev:0] └── [value] ArrayAccessNode [type:integer, lev:1] </pre>

```

└─ [array] VarNode(name: 'arr') [tab_index:41, type:array,
lev:0]
└─ [index] VarNode(name: 'i') [tab_index:39, type:integer,
lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	ArrayAccessExpr	37	PROGRAM	VOID	-1	1	0	0
39	i	38	VARIABLE	INTEGER	-1	1	0	0
40	value	39	VARIABLE	INTEGER	-1	1	0	1
41	arr	40	VARIABLE	ARRAY	0	1	0	2

```
BTAB
idx last lpar psze vsze
  0   41   -1    0    7
  1   -1   -1    0    0
```

```
ATAB
idx xtyp      etyp      eref low high elsz size
  0 INTEGER    INTEGER    21   1   5    1    5
```

Output Program

```
=== INTERMEDIATE CODE ===
0 INT 0 10
1 LDA 0 5
2 LOD 0 3
3 LIT 0 1
4 OPR 0 3
5 OPR 0 2
6 LDI 0 0
7 STO 0 4
8 RET 0 0
```

Screenshot



```
026$ ./codegen test/codegen/decorated-ast/dast-tc12-array-access.txt
=== INTERMEDIATE CODE ===
0 INT 0 10
1 LDA 0 5
2 LOD 0 3
3 LIT 0 1
4 OPR 0 3
5 OPR 0 2
6 LDI 0 0
7 STO 0 4
8 RET 0 0
```

Penjelasan

Kasus ini memverifikasi mekanisme indirect addressing untuk akses array. Ukuran frame INT 0 10 terdiri atas tiga slot header, variabel *i* (1 slot), value (1 slot), dan *arr* bertipe array[1..5] yang menempati 5 slot kontigu. Perhitungan ukuran yang benar penting agar elemen array tidak menempa lokasi memori lain.

Pada operasi `value := arr[i]`, alamat elemen dihitung saat runtime. Instruksi LDA 0 5 memuat alamat basis array, LOD 0 3 memuat nilai indeks *i*, dan LIT 0 1 memuat batas bawah array. Selanjutnya, OPR 0 3 (SUB) menghitung *i* - 1 untuk memperoleh offset, lalu OPR 0 2 (ADD) menambahkan offset tersebut ke alamat basis

sehingga diperoleh alamat elemen yang diakses.

Karena operasi ini merupakan pembacaan (rvalue), instruksi LDI digunakan untuk mengambil nilai dari alamat yang telah dihitung dan mendorongnya ke stack. Nilai tersebut kemudian disimpan ke variabel value melalui STO 0 4.

Kasus ini juga menegaskan peran LDI dan STI dalam mendukung akses memori tak langsung, sekaligus memastikan alamat yang digunakan berada dalam batas memori yang valid saat program dijalankan.

3.14 Test Case 13

Kode Program Input

```
program RecordAccessExpr;
type
  Point == record
    x, y: integer;
  end;
var
  p: Point;
  result: integer;
begin
  result := p.y;
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'RecordAccessExpr') [type:void, lev:0]
├── [decl] TypeDeclNode(name: 'Point') [type:void]
│   ├── [type] RecordTypeNode [tab_index:1, type:record, lev:0]
│   │   └── [field] VarDeclNode(names: 'x', 'y') [type:void, lev:1]
│   │       └── [type] VarNode(name: 'integer') [tab_index:21,
type:integer, lev:0]
│   └── [decl] VarDeclNode(names: 'p') [type:void, lev:0]
│       └── [type] VarNode(name: 'Point') [tab_index:41, type:record,
lev:0]
├── [decl] VarDeclNode(names: 'result') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
└── [body] BlockNode(btab_index: 2) [type:void, lev:1]
    ├── [stmt] AssignNode [type:void, lev:1]
    │   ├── [target] VarNode(name: 'result') [tab_index:43,
type:integer, lev:0]
    │   └── [value] RecordAccessNode(field: 'y') [tab_index:40,
type:integer, lev:1]
    │       └── [record] VarNode(name: 'p') [tab_index:42, type:record,
lev:0]
```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	RecordAccessExpr	37	PROGRAM	VOID	-1	1	0	0
39	x	-1	VARIABLE	INTEGER	-1	1	1	0
40	y	39	VARIABLE	INTEGER	-1	1	1	1
41	Point	38	TYPE	RECORD	1	1	0	0
42	p	41	VARIABLE	RECORD	1	1	0	0
43	result	42	VARIABLE	INTEGER	-1	1	0	2

BTAB

idx	last	lpar	psze	vsze
0	43	-1	0	3

1	40	-1	0	2
2	-1	-1	0	0

ATAB

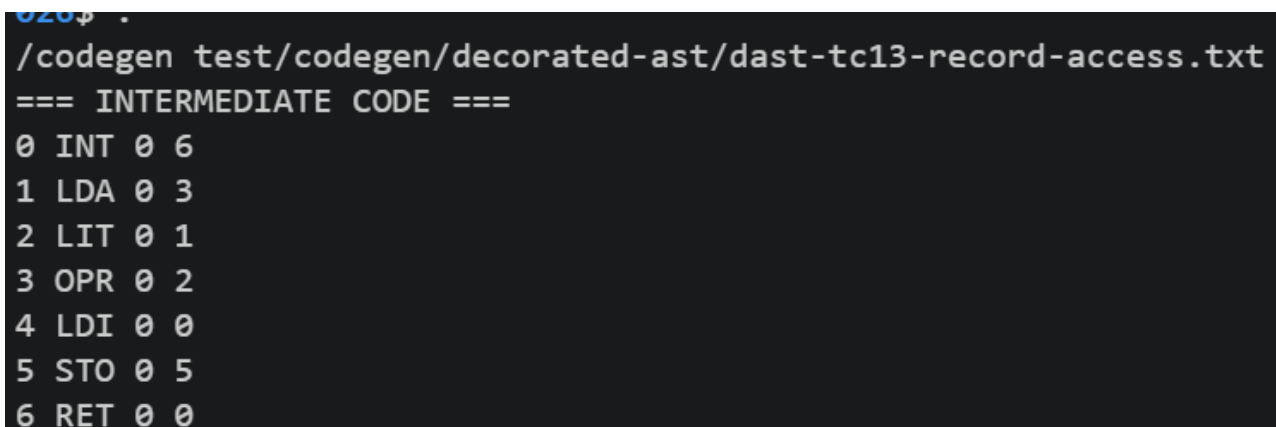
idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```
0 INT 0 6
1 LDA 0 3
2 LIT 0 1
3 OPR 0 2
4 LDI 0 0
5 STO 0 5
6 RET 0 0
```

Screenshot



```
026$ .
/codegen test/codegen/decorated-ast/dast-tc13-record-access.txt
=== INTERMEDIATE CODE ===
0 INT 0 6
1 LDA 0 3
2 LIT 0 1
3 OPR 0 2
4 LDI 0 0
5 STO 0 5
6 RET 0 0
```

Penjelasan

Kasus ini melengkapi pengujian array dengan struktur data komposit kedua, yaitu record. Mekanisme pengalamatannya mirip dengan array, tetapi offset setiap field bersifat statis dan telah diketahui saat kompilasi karena nama field sudah ditentukan.

Pada contoh ini, tipe Point memiliki dua field, x dan y, yang masing-masing memiliki offset 0 dan 1. Variabel p menempati dua slot kontigu (alamat 3-4), sedangkan result berada di alamat 5, sehingga frame memiliki ukuran INT 0 6.

Untuk operasi $\text{result} := p.y$, code generator membangkitkan LDA 0 3 untuk memuat alamat basis p, LIT 0 1 untuk memuat offset field y, dan OPR 0 2 (ADD) untuk menghitung alamat absolut field tersebut. Instruksi LDI kemudian digunakan untuk mengambil nilai pada alamat yang dihasilkan, dan STO 0 5 menyimpan nilai tersebut ke variabel result.

Berbeda dengan array yang memerlukan perhitungan offset berdasarkan indeks saat runtime, offset field pada record bersifat tetap sehingga dapat langsung digunakan. Kasus ini juga memverifikasi bahwa ukuran slot untuk tipe komposit dihitung dengan benar; tanpa perhitungan tersebut, lokasi result dapat bertumpang tindih dengan field milik p.

3.15 Test Case 14

Kode Program Input

```
program FunctionCallExpr;
var
  x, y, z: integer;
function sum(a, b: integer): integer;
begin
  sum := a + b;
end;
begin
  z := sum(x, y);
end.
```

Decorated AST input

```
=== DECORATED AST ===
ProgramNode(name: 'FunctionCallExpr') [type:void, lev:0]
├── [decl] VarDeclNode(names: 'x', 'y', 'z') [type:void, lev:0]
│   └── [type] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
├── [decl] FuncDeclNode(name: 'sum') [type:void]
│   ├── [param] ParamGroupNode(var: false, names: 'a', 'b') [type:void]
│   │   └── [type] VarNode(name: 'integer') [tab_index:21,
type:integer, lev:0]
│   └── [returns] VarNode(name: 'integer') [tab_index:21, type:integer,
lev:0]
│   └── [body] BlockNode [type:void, lev:1]
│       └── [stmt] AssignNode [type:void, lev:1]
│           ├── [target] VarNode(name: 'sum') [tab_index:42,
type:integer, lev:0]
│           └── [value] BinOpNode(op: '+') [type:integer, lev:1]
│               ├── [left] VarNode(name: 'a') [tab_index:43,
type:integer, lev:1]
│               └── [right] VarNode(name: 'b') [tab_index:44,
type:integer, lev:1]
└── [body] BlockNode(btab_index: 2) [type:void, lev:1]
    └── [stmt] AssignNode [type:void, lev:1]
        ├── [target] VarNode(name: 'z') [tab_index:41, type:integer,
lev:0]
        └── [value] FuncCallExprNode(name: 'sum') [tab_index:42,
type:integer, lev:1]
```

```

└─ [arg] VarNode(name: 'x') [tab_index:39, type:integer,
lev:0]
└─ [arg] VarNode(name: 'y') [tab_index:40, type:integer,
lev:0]

```

=== SYMBOL TABLE ===

TAB

idx	id	link	obj	type	ref	nrm	lev	adr
0	and	-1	RESERVED	VOID	-1	1	0	0
1	array	0	RESERVED	VOID	-1	1	0	0
2	begin	1	RESERVED	VOID	-1	1	0	0
3	case	2	RESERVED	VOID	-1	1	0	0
4	const	3	RESERVED	VOID	-1	1	0	0
5	div	4	RESERVED	VOID	-1	1	0	0
6	downto	5	RESERVED	VOID	-1	1	0	0
7	do	6	RESERVED	VOID	-1	1	0	0
8	else	7	RESERVED	VOID	-1	1	0	0
9	end	8	RESERVED	VOID	-1	1	0	0
10	for	9	RESERVED	VOID	-1	1	0	0
11	function	10	RESERVED	VOID	-1	1	0	0
12	if	11	RESERVED	VOID	-1	1	0	0
13	mod	12	RESERVED	VOID	-1	1	0	0
14	not	13	RESERVED	VOID	-1	1	0	0
15	of	14	RESERVED	VOID	-1	1	0	0
16	or	15	RESERVED	VOID	-1	1	0	0
17	procedure	16	RESERVED	VOID	-1	1	0	0
18	program	17	RESERVED	VOID	-1	1	0	0
19	record	18	RESERVED	VOID	-1	1	0	0
20	repeat	19	RESERVED	VOID	-1	1	0	0
21	integer	20	TYPE	INTEGER	-1	1	0	1
22	real	21	TYPE	REAL	-1	1	0	1
23	boolean	22	TYPE	BOOLEAN	-1	1	0	1
24	char	23	TYPE	CHAR	-1	1	0	1
25	string	24	TYPE	STRING	-1	1	0	1
26	then	25	RESERVED	VOID	-1	1	0	0
27	to	26	RESERVED	VOID	-1	1	0	0
28	type	27	RESERVED	VOID	-1	1	0	0
29	until	28	RESERVED	VOID	-1	1	0	0
30	var	29	RESERVED	VOID	-1	1	0	0
31	while	30	RESERVED	VOID	-1	1	0	0
32	true	31	CONSTANT	BOOLEAN	-1	1	0	1
33	false	32	CONSTANT	BOOLEAN	-1	1	0	0
34	writeln	33	PROCEDURE	VOID	-1	1	0	0
35	readln	34	PROCEDURE	VOID	-1	1	0	0
36	write	35	PROCEDURE	VOID	-1	1	0	0
37	read	36	PROCEDURE	VOID	-1	1	0	0
38	FunctionCallExpr	37	PROGRAM	VOID	-1	1	0	0
39	x	38	VARIABLE	INTEGER	-1	1	0	0
40	y	39	VARIABLE	INTEGER	-1	1	0	1
41	z	40	VARIABLE	INTEGER	-1	1	0	2
42	sum	41	FUNCTION	INTEGER	1	1	0	0

43	a	-1	VARIABLE	INTEGER	-1	1	1	0
44	b	43	VARIABLE	INTEGER	-1	1	1	1

BTAB

idx	last	lpar	psze	vsze
0	42	-1	0	3
1	44	44	2	0
2	-1	-1	0	0

ATAB

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

Output Program

=== INTERMEDIATE CODE ===

```

0 JMP 0 10
1 INT 0 6
2 STO 1 4
3 STO 1 3
4 LOD 1 3
5 LOD 1 4
6 OPR 0 2
7 STO 1 5
8 LOD 1 5
9 RET 0 0
10 INT 0 6
11 LOD 0 3
12 LOD 0 4
13 CAL 0 1
14 STO 0 5
15 RET 0 0

```

Screenshot

```

naufai@LAPTOP-B15ASLPI:/mnt/c/Users/Naufai/OneDrive/Documents/SEMESTER_4/TBF0/Milestone4/TBS-Tubes-1F2224-026$ ./codegen test/codegen/decorated-ast/dast-tc14-function-call-expression.txt
=== INTERMEDIATE CODE ===
0 JMP 0 10
1 INT 0 6
2 STO 1 4
3 STO 1 3
4 LOD 1 3
5 LOD 1 4
6 OPR 0 2
7 STO 1 5
8 LOD 1 5
9 RET 0 0
10 INT 0 6
11 LOD 0 3
12 LOD 0 4
13 CAL 0 1
14 STO 0 5
15 RET 0 0

```

Penjelasan
Kasus ini merupakan pengujian paling komprehensif karena mencakup penempatan subprogram, pengikatan parameter, dan mekanisme nilai balik fungsi. Program diawali dengan JMP 0 10 pada baris 0 untuk melewati badan fungsi sehingga eksekusi dimulai dari program utama di baris 10. Sementara itu, badan fungsi sum ditempatkan pada baris 1–9. Prologue fungsi dimulai dengan INT 0 6 yang mengalokasikan frame berisi 3 slot header, 2 parameter (a dan b pada alamat 3-4), serta 1 slot hasil pada alamat 5. Instruksi STO 1 4 dan STO 1 3 kemudian mengikat argumen yang telah didorong pemanggil ke stack ke slot parameter fungsi. Karena stack bersifat LIFO, argumen terakhir disimpan terlebih dahulu ke b, kemudian argumen sebelumnya ke a. Badan fungsi sum := a + b dibangkitkan pada baris 4–7. Karena Arion tidak memiliki kata kunci return, penugasan ke nama fungsi diterjemahkan sebagai penyimpanan nilai ke slot hasil (STO 1 5). Sebelum keluar dari fungsi, LOD 1 5 memuat kembali nilai tersebut ke stack sehingga tersedia bagi pemanggil, lalu eksekusi diakhiri dengan RET. Pada program utama, pemanggilan z := sum(x, y) diterjemahkan dengan memuat argumen x dan y ke stack, menjalankan CAL 0 1 untuk memanggil fungsi, lalu menyimpan nilai balik yang tersisa di stack ke z menggunakan STO 0 5. Kasus ini memverifikasi keseluruhan mekanisme pemanggilan fungsi, mulai dari pembentukan frame, pengikatan parameter, eksekusi badan fungsi, hingga pengembalian nilai, yang menjadi dasar bagi implementasi fungsi dan rekursi.

3.16 Pengujian Unit Interpreter

Folder test/interpreter/ berisi pengujian yang menyorot instruksi stack machine secara spesifik, memastikan tiap kelompok instruksi (LIT/LOD/STO, OPR, JMP/JPC, CAL/RET, WRT/WRTLN) dieksekusi dengan benar.

Berkas	Instruksi yang diuji	Keluaran
tc1-lit-lod-sto	LIT, LOD, STO (muat/simpan nilai)	25
tc2-opr-arithmetic	OPR ADD/MUL/SUB/IDIV dengan presedensi	29
tc3-opr-comparison	OPR GTR/LEQ dan AND (boolean)	true
tc4-jmp-jpc	JMP, JPC (perulangan kendali)	5
tc5-cal-ret	CAL, RET (pemanggilan subprogram)	42
tc6-wrt-wrtln	WRT (tanpa newline) dan WRTLN	78

3.17 Pengujian Integrasi Pipeline Penuh

Ke-14 kasus di atas memverifikasi Intermediate Code hasil code generation. Untuk memvalidasi eksekusi penuh oleh Virtual Machine, disediakan tujuh program lengkap pada folder

test/integration/ yang dijalankan dari source code Arion hingga keluaran akhir. Setiap berkas ekspektasi memuat Intermediate Code beserta output akhir sehingga kedua aspek terverifikasi sekaligus.

Berkas	Fitur utama yang diuji	Keluaran
int1-arithmetic	Aritmetika, presedensi operator, div, writeln	32
int2-control-flow	for bersarang dengan if dan mod (jumlah genap 1..10)	30
int3-array	Tulis & baca <i>array</i> di dalam perulangan	55
int4-record	Akses <i>field</i> record dan aritmetika ($3^2 + 4^2$)	25
int5-function	Pemanggilan fungsi dengan if-else dan nilai balik	42
int6-recursion	Rekursi ganda (Fibonacci fib(10))	55
int7-writeln-multi	writeln multi-argumen (<i>string</i> + <i>integer</i>)	Nilai = 100

3.18 Pengujian Bonus Runtime Vulnerabilities

Disediakan empat pengujian negatif pada folder test/interpreter/ yang sengaja memicu kondisi berbahaya. Hasil yang diharapkan adalah Virtual Machine mendeteksi dan menghentikan eksekusi dengan pesan error informatif, tanpa membuat program induk crash, dan dengan Intermediate Code tetap tercetak

Berkas	Kerentanan	Hasil (pesan error)
tc7-division-by-zero	Pembagian bilangan bulat dengan nol	<i>Division by zero: integer division by zero</i>
tc8-integer-overflow	Penjumlahan melampaui INT_MAX	<i>Invalid operation: integer addition overflow</i>
tc9-out-of-bounds	Akses <i>array</i> di luar batas (arr[99])	<i>Invalid memory access: indirect store out of bounds</i>
tc10-stack-overflow	Rekursi tak hingga (tanpa kasus basis)	<i>Stack overflow: call stack exceeded configured frame limit</i>

Bab V

Kesimpulan dan Saran

5.1 Kesimpulan

Implementasi Milestone 4 berhasil menyelesaikan keseluruhan alur kompilasi bahasa Arion, mulai dari source code hingga tahap eksekusi. Code generator berbasis DFS mampu mentransformasikan Decorated AST menjadi intermediate code berarsitektur stack machine yang sesuai untuk seluruh konstruksi bahasa. Selanjutnya, interpreter berbasis stack machine dapat mengeksekusi kode tersebut dengan benar sekaligus mengelola memori melalui mekanisme stack frame.

Fungsionalitas yang telah terimplementasi mencakup evaluasi ekspresi aritmetika, boolean, dan relasional; seluruh struktur kendali alur program seperti if, while, for, repeat, dan case; dukungan terhadap struktur data array dan record; serta pemanggilan procedure dan function lengkap dengan parameter, nilai balik, dan rekursi. Selain itu, sistem juga mampu mendeteksi dan menangani berbagai jenis runtime error.

Keberhasilan implementasi ini dibuktikan melalui pengujian otomatis, di mana seluruh kasus uji berhasil dijalankan dan menghasilkan keluaran yang sesuai dengan spesifikasi.

5.2 Saran

1. Menambahkan fase optimasi pada modul codegen sebelum intermediate code dieksekusi oleh Virtual Machine.
2. Sistem dapat dikembangkan agar mendukung alokasi memori dinamis di heap yang dilengkapi dengan garbage collector otomatis
3. Menambahkan fitur tracer atau debugger mode pada interpreter yang memungkinkan pengguna melihat perubahan isi komponen evaluation stack, nilai runtime variabel di setiap alamat memori, serta pergerakan Instruction Pointer (IP) secara step-by-step

Referensi

GeeksforGeeks. *Phases of a Compiler*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/phases-of-a-compiler/>

GeeksforGeeks. *Abstract Syntax Tree (AST) in Compiler Design*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/abstract-syntax-tree-ast-in-compiler-design/>

GeeksforGeeks. *Symbol Table in Compiler Design*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/symbol-table-compiler/>

GeeksforGeeks. *Intermediate Code Generation in Compiler Design*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/intermediate-code-generation-in-compiler-design/>

GeeksforGeeks. *Three Address Code in Compiler Design*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/three-address-code-compiler/>

GeeksforGeeks. *Compiler vs Interpreter*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/compiler-vs-interpreter-2/>

GeeksforGeeks. *Fetch-Decode-Execute Cycle*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/different-instruction-cycles/>

GeeksforGeeks. *Stack Data Structure*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/stack-data-structure/>

GeeksforGeeks. *Stack Frame in Computer Organization*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/stack-frame-in-computer-organization/>

GeeksforGeeks. *Type Checking in Compiler Design*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/type-checking-in-compiler-design/>

GeeksforGeeks. *Stack Overflow in Programming*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/stack-overflow-in-programming/>

GeeksforGeeks. *Buffer Overflow Attack with Example*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/buffer-overflow-attack-with-example/>

GeeksforGeeks. *Array Index Out of Bounds Exception*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/array-index-out-of-bounds-exception-in-java/>

GeeksforGeeks. *Overflow in Arithmetic*. Diakses pada 2 Juni 2026.

<https://www.geeksforgeeks.org/overflow-in-arithmetic/>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Addison Wesley.

Lampiran

A. Pranala Repository Github

<https://github.com/tmthyberd/TBS-Tubes-IF2224-2026>

B. Pembagian Tugas

NIM	Deskripsi Tugas	Persentase
13524012	Laporan test-case, vm operations, codegen main, interpreter, testing dan codegen statement	25%
13524076	Melengkapi laporan, mengimplementasikan Codegen Expression & OPR Mapper	25%
13524078	Membuat template Laporan, laporan bab 1, dan menyiapkan test case untuk di uji, membuat diagram	25%
13524092	Laporan bab 2, fondasi backend compiler: interface codegen, format instruksi, memori VM, nilai runtime, interface VM, dan exception runtime.	25%

C. Gambar Diagram

 Class Diagram dan FlowChart